



بسمه تعالی

# آزمایشگاه مخابرات دیجیتال

استاد: دکتر علی الفت

ویرایش ۱/۷ (آزمایشی)

## فهرست مطالب آزمایشگاه مخابرات دیجیتال

|    |                                                                |
|----|----------------------------------------------------------------|
| ۳  | ..... مقدمه: معرفی آزمایشگاه مخابرات دیجیتال                   |
| ۵  | ..... آزمایش اول: رادیونرم افزار و پایش طیف سیگنال های رادیویی |
| ۱۰ | ..... آزمایش دوم: معرفی مدولاسیون دیجیتال خطی                  |
| ۲۰ | ..... آزمایش سوم: کاوش در مدولاسیون دیجیتال خطی                |
| ۲۷ | ..... آزمایش چهارم: مدولاسیون FSK همدوس                        |
| ۳۳ | ..... آزمایش پنجم: آشکارسازی ناهمدوس                           |
| ۴۱ | ..... آزمایش ششم: انتقال دیجیتال از درون کانال باندمحدود AWGN  |

## مقدمه

## معرفی آزمایشگاه مخابرات دیجیتال

## رویکرد فعالیت‌های آزمایشگاه مخابرات دیجیتال

به عنوان مقدمه، مطالبی در مورد رویکرد فعالیت‌های آزمایشگاه مخابرات دیجیتال ارائه می‌شود. آزمایشگاه مخابرات دیجیتال، در این ترم شامل ۱۲ جلسه می‌باشد. دانشجوی در طول این ۱۲ جلسه با مفاهیم مخابرات دیجیتال، ابزار شبیه‌سازی و یک سخت‌افزار آموزشی این حوزه آشنا می‌شود.

عمده‌ی آزمایشگاه‌هایی که در حوزه‌ی مخابرات سیستم برگزار می‌گردد، بر مبنای شبیه‌سازی است. این امر باعث می‌شود دانشجوی با چالش‌های پیاده‌سازی عملی الگوریتم‌ها آشنا نشود. علاوه بر این معمولاً سخت‌افزارهایی که پیاده‌سازی‌های عملی بر روی آن صورت می‌پذیرد، دارای پیچیدگی بالایی است. از این رو می‌بایست مدت زمان قابل توجهی صرف راه‌اندازی و آموزش این سخت‌افزارها شود. این امر باعث می‌شود فرصتی برای آموزش مفاهیم حوزه‌ی مخابرات دیجیتال باقی نماند.

مطلب حائز اهمیت دیگر آن است که دانشجویهایی که وارد این آزمایشگاه می‌شوند، تسلط کاملی نسبت به ابزارهای شبیه‌سازی ندارند و مدت زمان قابل توجهی در طول آزمایش‌ها، صرف آموزش این ابزارها می‌شود. بنابراین می‌بایست یک بسته‌ی آموزشی برای تسلط بر روی ابزارهای شبیه‌سازی تهیه شود. به منظور کاهش زمان این آموزش، می‌بایست تمرکز بر روی یک ابزار شبیه‌سازی باشد و راه‌اندازی سخت‌افزار و شبیه‌سازی‌های متکی بر آن با استفاده از همین یک ابزار انجام شود.

با استدلال‌های ذکر شده طراحی این آزمایشگاه مبتنی بر نرم‌افزار MATLAB و تنها اختصاص به بخش کدنویسی آن دارد. به علت محدودیت زمان، امکان آموزش بخش Simulink این نرم‌افزار نیز وجود ندارد. سخت‌افزاری که برای بخش عملی آزمایشگاه مورد استفاده قرار می‌گیرد، رادیو نرم‌افزار ADALM-PLUTO شرکت Analog Devices می‌باشد که در شکل ۱ نشان داده شده است. این رادیو نرم‌افزار را می‌توان به راحتی با برنامه‌ی MATLAB کنترل نمود و از آن برای دریافت و ارسال سیگنال استفاده نمود و یک سامانه‌ی کامل مخابرات دیجیتال را پیاده‌سازی کرد.



شکل ۱ رادیو نرم‌افزار ADALM-PLUTO

با توجه به شرایط آموزشی فعلی و برگزاری آزمایشگاه به صورت مجازی، بخش کار با سخت افزار کم رنگ شده است و تنها ویدیوهایی برای آشنایی با سخت افزار و ملاحظات عملی پیاده سازی ارایه می گردد. فعالیت های این آزمایشگاه شامل سه بخش تمرین های قبل از آزمایشگاه، شبیه سازی درون آزمایشگاه و نمایش عملکرد واقعی با رادیو نرم افزار می باشد. این روند در شکل ۲ نشان داده شده است.



شکل ۲ روند فعالیت های آزمایشگاه مخابرات دیجیتال

در این روند به صورت تدریجی پایه های هر مرحله در مرحله ی قبل چیده می شود تا آزمایش با موفقیت و با بیشترین بازده انجام شود. با این رویکرد، روند پیشرفت آزمایش ها با سرعت بالاتری انجام شده و دانشجو بر روی مطالب و مفاهیم مطرح شده در هر آزمایش تسلط بیشتری کسب خواهد نمود.

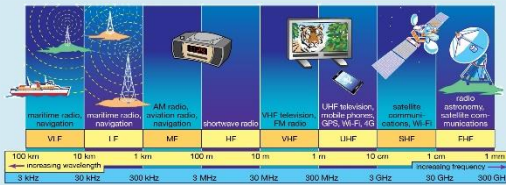
به منظور تسلط بر مباحث تئوری ارایه شده در درس های گذشته نظیر مخابرات دیجیتال و آمادگی برای شبیه سازی آزمایش ها، تمرین هایی به عنوان فعالیت قبل از کلاس در نظر گرفته شده است. دانشجویان با انجام تمرین ها قبل از آزمایشگاه، با مفاهیم کلیدی هر آزمایش آشنا می شوند. زمانی که پایه های تئوری هر آزمایش بنا شد، دانشجویان موظفند شبیه سازی های مربوط به طراحی و پیاده سازی اجزای یک سامانه ی مخابرات دیجیتال را انجام دهند و نتیجه ی کار را در محیط شبیه سازی نرم افزار مهندسی MATLAB ارزیابی و مشاهده نمایند. منطق این شبیه سازی ها آن است که دانشجو رفتار عملکردی اجزای یک سامانه ی مخابرات دیجیتال را در یک محیط کنترل شده مشاهده نماید. در نتیجه دانشجو آمادگی کافی جهت ورود به دنیای واقعی با عوامل غیر ایده آل فراوان را کسب می نماید. زمانی که مجموعه ی مهارت های دانشجویان با آمادگی تئوریک و شبیه سازی تقویت گردید، آمادگی لازم برای ورود به پیاده سازی اجزای سامانه ی مخابرات دیجیتال در یک محیط بی سیم واقعی به دست می آید. دانشجو در این مرحله با استفاده از ابزارهایی که نرم افزار MATLAB در اختیار می گذارد، عملکرد رادیو نرم افزار را مشاهده می نماید.

## شیوه ی ارزیابی آزمایشگاه

در ادامه سهم هر یک از فعالیت های آزمایشگاه در نمره ی پایانی آمده است. دقت نمایید که همه ی مراحل به صورت انفرادی می باشد.

- تمرین های قبل از آزمایش: ۶ نمره
- انجام آزمایش ها: ۸ نمره
- گزارش کار آزمایشگاه: ۲ نمره
- پروژه ی پایانی یا آزمون: ۴ نمره
- حضور و غیاب: هر جلسه غیبت کسر ۱ نمره

در این ترم دستورکار آزمایشگاه در اختیار دانشجو قرار می گیرد. در روز آزمایشگاه توضیحاتی در مورد آزمایش ارایه می شود. هر کدام از گروه ها در طول سه ساعت آزمایشگاه با همیاران آموزشی آزمایشگاه در ارتباط هستند و به کمک آن ها کدهای خود را رفع اشکال می نمایند. در پایان روز ارایه ی درس، تمرین های قبل از آزمایش برای هفته ی آینده بارگزاری شده و دانشجویان می بایست تا یک روز قبل از شروع کلاس بعد جواب های آن را در سایت بارگزاری نمایند.



# آزمایش اول

## معادل باندپایه و میان گذر

### اهداف آزمایش

طیف رادیویی به بخشی از طیف الکترومغناطیس در محدوده‌ی فرکانسی 3kHz تا 300GHz گفته می‌شود. از طیف رادیویی به صورت گسترده در سرویس‌های مخابراتی نظیر ارسال سیگنال‌های تلویزیون، رادیو، موبایل، WiFi و سامانه‌های ناوبری و آشکارسازی نظیر رادار، GPS، دیده‌بانی رادیویی، فرستنده‌ها و ... استفاده می‌شود. در این آزمایش استفاده از سخت‌افزار ADALM-PLUTO شروع می‌شود و بناست طیف فرکانسی محدوده‌ی 25MHz تا 6GHz مورد پایش قرار بگیرد. آزمایش‌هایی که صورت می‌پذیرد کمک می‌کند تا این بخش از طیف مورد کنکاش قرار بگیرد و دانشجو به جست‌جو و شناسایی سیگنال‌های رادیویی در این محدوده‌ی فرکانسی بپردازد. از جمله مفاهیمی که در رادیو نرم‌افزارها و سامانه‌های مخابراتی مورد استفاده قرار می‌گیرد، مفهوم معادل پایین‌گذر و میان‌گذر سیگنال است. دانشجو در این آزمایش با این مفاهیم آشنایی کامل پیدا می‌کند.

در انتهای این آزمایش دانشجو می‌بایست:

- تسلط کامل بر روی مفاهیم معادل پایین‌گذر و بالاگذر داشته باشد و بتواند این دو معادل را به یکدیگر تبدیل نماید.
- با طیف فرکانسی محدوده‌ی 25MHz تا 6GHz آشنا باشد و درک مناسبی از سیگنال‌های موجود در این گستره‌ی فرکانسی داشته باشد.
- آشنایی کامل با بسته‌ی رادیو نرم‌افزار ADALM-PLUTO داشته باشد و اتصالات کابل‌ها و آنتن‌ها را به رادیو نرم‌افزار انجام دهد.
- آشنایی ضمنی با نحوه‌ی افزودن این رادیو نرم‌افزارها به نرم‌افزار MATLAB را داشته باشد.
- با ساختار رادیو نرم‌افزارهای ADALM-PLUTO آشنایی کافی داشته باشد.
- بتواند پارامترهای رادیو نرم‌افزارها را با استفاده از یک برنامه‌ی MATLAB تنظیم نماید و رادیو نرم‌افزارها را در فرکانس دلخواه تیون نماید.
- بتواند داده‌های خروجی رادیو نرم‌افزار را دریافت نموده و طیف آن را به صورت به‌لحظه رسم نماید.
- سیگنال‌های رادیو FM، سیگنال‌های تلفن همراه نسل دوم، سوم و چهارم، سیگنال ریموت درب، سیگنال صوت و تصویر دیجیتال را پایش نموده و به ویژگی‌های طیف فرکانسی مسلط باشد.

### ابزارهای مورد نیاز

رایانه

نرم‌افزار MATLAB R2020b

- MathWorks DSP System Toolbox
- MathWorks Communications System Toolbox
- MathWorks Signal Processing Toolbox

رادیو نرم‌افزار ADALM-PLUTO

کابل کوکسیال و آنتن



## تئوری

## معادل باندپایه‌ی مختلط سیگنال‌های میان‌گذر

تقریباً همه‌ی سامانه‌های مخابراتی با مدوله کردن یک شکل موج حاوی اطلاعات بر روی یک سیگنال حامل سینوسی پیاده‌سازی می‌شوند. فرکانس حامل سیگنال ارسالی حاوی هیچ اطلاعاتی نیست و در واقع سیگنال مدوله‌کننده‌ی حامل در برگیرنده‌ی اطلاعات است. از این رو به توصیف سیگنال‌های مخابراتی، مستقل از فرکانس حامل نیاز است. در نتیجه‌ی این نیاز، مهندسين سامانه‌های مخابراتی برای سادگی کارها از معادل باندپایه‌ی مختلط<sup>۱</sup> یا معادل پایین‌گذر سیگنال‌های مخابراتی استفاده می‌نمایند. تمام سامانه‌های مخابراتی می‌تواند با بیان باندپایه‌ی مختلط تحلیل شود و معمولاً نیز چنین کاری صورت می‌پذیرد. یکی از مزایای معادل باندپایه‌ی مختلط سادگی است. همه‌ی سیگنال‌ها پایین‌گذر هستند و ایده‌های پایه‌ای مدولاسیون و پردازش سیگنال مخابراتی به سادگی توسعه می‌یابد. علاوه بر این عمده‌ی گیرنده‌هایی که شکل موج‌های دریافتی را به صورت دیجیتال پردازش می‌نمایند، از معادل باندپایه‌ی مختلط برای توسعه‌ی الگوریتم‌های پردازش استفاده می‌نمایند.

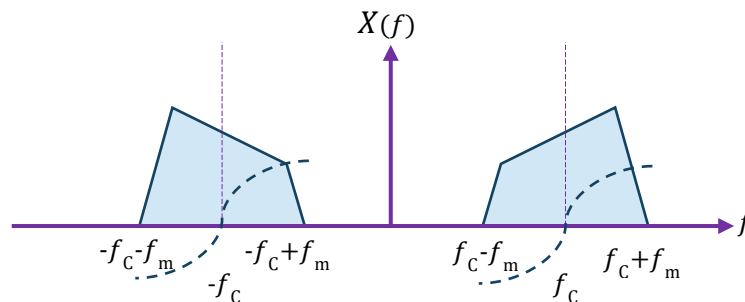
گام اول در بسط معادل باندپایه‌ی مختلط تعریف سیگنال باندپایه است.

تعریف کیفی سیگنال میان‌گذر: سیگنال میان‌گذر  $x(t)$  سیگنالی است که مرکز طیف انرژی یک طرفه‌ی آن در فرکانس غیرصفر  $f_c$  باشد و دنباله‌ی این طیف تا فرکانس صفر ادامه ندارد.

تعریف ریاضی سیگنال میان‌گذر: به عبارت دیگر سیگنالی میان‌گذر است که حقیقی باشد و تبدیل فوریه‌ی آن دارای این ویژگی است

$$\forall f: |f| - f_c > f_m \rightarrow X(f) = 0 \text{ و } f_c > f_m \text{ باشد}$$

اگر پهنای باند دوطرفه‌ی انتقال سیگنال میان‌گذر برابر با  $2f_m$  هرتز باشد، طیف یک طرفه‌ی سیگنال میان‌گذر تنها در بازه‌ی فرکانسی  $[f_c - f_m, f_c + f_m]$  غیرصفر می‌باشد. این مسأله بدان معناست که سیگنال‌های میان‌گذر دارای قید  $f_m < f_c$  می‌باشند. شکل ۱ نمونه‌ای از یک طیف میان‌گذر را نشان می‌دهد. از آن جا که یک سیگنال میان‌گذر  $x_c(t)$  سیگنالی است که به صورت فیزیکی قابل تحقق است، سیگنال حقیقی است و انرژی سیگنال نسبت به فرکانس  $f = 0$  متقارن است. اندازه‌ی نسبی  $B_T$  و  $f_c$  اهمیت نداشته و تنها می‌بایست طیف سیگنال مقدار ناچیزی در حول فرکانس DC داشته باشد. در مخابرات تلفنی این ناحیه‌ی طیف، ناچیز و در حدود 300Hz است در حالی که در مخابرات ماهواره‌ای این مقدار می‌تواند چندین گیگاهرتز باشد.



شکل ۱ پاسخ فرکانسی یک سیگنال حقیقی میان‌گذر

یک سیگنال میان‌گذر دارای بیانی به صورت زیر است.

$$x(t) = x_i(t) \cos 2\pi f_c t - x_q(t) \sin 2\pi f_c t \quad (۱)$$

$$= x_a(t) \cos(2\pi f_c t + x_p(t)) \quad (۲)$$

در رابطه‌ی فوق  $f_c$  فرکانس حامل بوده که در محدوده‌ی  $f_c - f_m \leq f_c \leq f_c + f_m$  قرار دارد. معمولاً به سیگنال  $x_i(t)$  در معادله‌ی (۱) مولفه‌ی هم‌فاز سیگنال و به سیگنال  $x_q(t)$  مولفه‌ی غیرهم‌فاز سیگنال میان‌گذر گفته می‌شود.  $x_i(t)$  و  $x_q(t)$  سیگنال‌های پایین‌گذر با مقادیر حقیقی هستند که طیف یک طرفه‌ی انرژی آن در فرکانس‌های بالاتر از  $2f_m$  دارای مقدار ناچیزی است. می‌بایست توجه کرد که فرکانس مرکزی سیگنال میان‌گذر  $f_c$  و فرکانس حامل  $f_c$  همواره با یکدیگر یکسان نیستند. در حالی که  $f_c$  به صورت تئوری می‌تواند زنجیره‌ای از مقادیر را اختیار کند، مقدار بدیهی  $f_c$  می‌تواند منجر به ساده‌ترین بیان شود. برای سیگنال حامل معمولاً یک عبارت کسینوسی

<sup>۱</sup> Complex Baseband Representation



در نظر گرفته می شود. از این رو مولفه ی  $x_i(t)$  هم فاز با سیگنال حامل است. به صورت مشابه عبارت سینوسی به اندازه ی ۹۰ درجه با کسینوس یا عبارت سیگنال حامل اختلاف فاز دارد و از این رو مولفه ی  $x_q(t)$  با حامل غیرهم فاز می باشد. معادله ی (۱) با عنوان فرم کانونی سیگنال میان گذر شناخته می شود. معادله ی (۲) صورت دامنه و فاز سیگنال میان گذر است که در آن  $x_a(t)$  دامنه ی سیگنال و  $x_p(t)$  فاز سیگنال است. سیگنال میان گذر دارای دو درجه ی آزادی است و بیان های کانونی  $I/Q$  با دامنه و فاز با یکدیگر معادل هستند. تبدیل بین این دو بیان به صورت زیر می باشد.

$$x_a(t) = \sqrt{x_i^2(t) + x_q^2(t)}, \quad x_p(t) = \tan^{-1}(x_q(t), x_i(t)) \quad (3)$$

$$x_i(t) = x_a(t) \cos x_p(t), \quad x_q(t) = x_a(t) \sin x_p(t) \quad (4)$$

می بایست دقت نمود که تابع تانژانت در معادله ی (۳) دارای برد  $[-\pi, \pi]$  می باشد و به عبارتی علامت هر دو  $x_i(t)$  و  $x_q(t)$  و نسبت  $x_q(t)$  و  $x_i(t)$  برای محاسبه ی تابع مورد نیاز است. جزییات تحلیل طراحی مخابراتی تعیین می کند که کدام صورت سیگنال باندپایه راحت تر به کار گرفته می شود.

یک سیگنال با مقادیر مختلط با نام پوش مختلط به صورت زیر تعریف می شود.

$$x_z(t) = \frac{1}{2}x_i(t) + \frac{j}{2}x_q(t) = \frac{x_a(t)}{2}e^{jx_p(t)} \quad (5)$$

سیگنال میان گذر اصلی را می توان به صورت زیر از روی پوش مختلط به دست آورد.

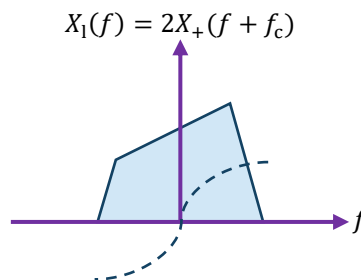
$$x(t) = \Re[2x_z(t)e^{j2\pi f_c t}] \quad (6)$$

از آن جا که تابع نمایی مختلط تنها فرکانس مرکزی را تعیین می نماید، سیگنال مختلط  $x_z(t)$  همه ی اطلاعات موجود در  $x(t)$  را در بر دارد. با استفاده از بیان باندپایه ی سیگنال های میان گذر، تحلیل های سامانه ی مخابراتی صورت ساده تری به خود می گیرد.

### تبدیل بین سیگنال میان گذر و معادل پایین گذر

در ادامه روشی برای تبدیل بین سیگنال میان گذر و معادل پایین گذر یا سیگنال پوش مختلط آورده شده است. سیگنال معادل پایین گذر که با  $x_1(t)$  نمایش داده می شود، دارای پاسخ فرکانسی به صورت زیر است که در شکل ۲ نشان داده شده است.

$$X_1(f) = 2X_+(f + f_c) \quad (7)$$

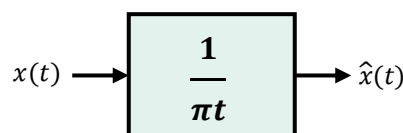


شکل ۲ پاسخ فرکانسی مربوط به معادل پایین گذر سیگنال شکل

می توان نشان داد که معادل پایین گذر یک سیگنال میان گذر را می توان با استفاده از رابطه ی زیر به دست آورد.

$$x_1(t) = (x(t) + j\hat{x}(t))e^{-j2\pi f_c t} \quad (8)$$

در رابطه ی فوق  $\hat{x}(t)$ ، تبدیل هیلبرت سیگنال  $x(t)$  است که در شکل ۳ نشان داده شده است.



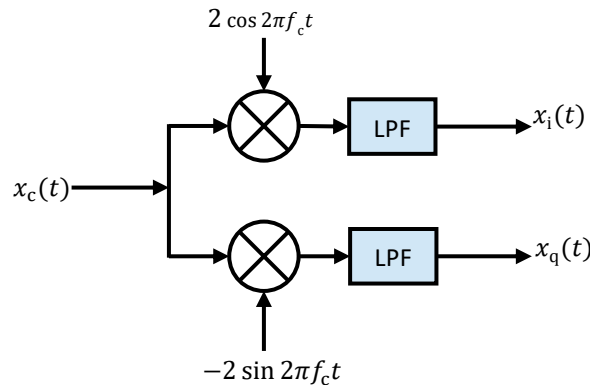
شکل ۳ تبدیل هیلبرت سیگنال  $x(t)$

با توجه به رابطه‌ی (۸)، سیگنال  $x_1(t)$  یک سیگنال مختلط خواهد بود. همان‌طور که اشاره شد، قسمت حقیقی و مختلط این سیگنال به ترتیب مولفه‌های هم‌فاز<sup>۲</sup> و غیرهم‌فاز<sup>۳</sup> نامیده شده و با  $x_i(t)$  و  $x_q(t)$  نشان داده می‌شود. بنابراین می‌توان نوشت:

$$x_1(t) = (x(t) + j\hat{x}(t))e^{-j2\pi f_c t} = x_i(t) + jx_q(t) \quad (9)$$

$$\begin{cases} x_i(t) = x(t) \cos(2\pi f_c t) + \hat{x}(t) \sin(2\pi f_c t) \\ x_q(t) = \hat{x}(t) \cos(2\pi f_c t) - x(t) \sin(2\pi f_c t) \end{cases} \quad (10)$$

از آنجا که فیلتر هیلبرت یک فیلتر غیرعلی است، در عمل برای به دست آوردن  $x_i(t)$  و  $x_q(t)$  نیازی به محاسبه‌ی تبدیل هیلبرت نیست و می‌توان از روشی مشابه شکل ۴ استفاده می‌شود. فیلتر پایین‌گذر می‌بایست سیگنال پوش مختلط را عبور دهد و مولفه‌های دو برابر فرکانس مرکزی را حذف کند.



شکل ۴ به دست آوردن معادل پایین‌گذر بدون استفاده از تبدیل هیلبرت

اگر معادل پایین‌گذر را داشته باشیم، با در نظر گرفتن رابطه‌ی (۹) می‌توان سیگنال میان‌گذر را به دست آورد. معادل میان‌گذر به صورت زیر به دست می‌آید.

$$\begin{cases} x(t) = \Re\{x_1(t)e^{j2\pi f_c t}\} = x_i(t) \cos 2\pi f_c t - x_q(t) \sin 2\pi f_c t \\ \hat{x}(t) = \Im\{x_1(t)e^{j2\pi f_c t}\} = x_q(t) \cos 2\pi f_c t + x_i(t) \sin 2\pi f_c t \end{cases} \quad (11)$$

### چگالی طیف توان فرآیند تصادفی ایستاد

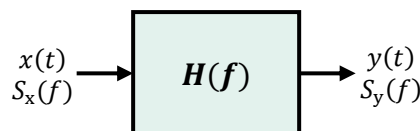
فرآیند  $x(t)$  ایستاد به مفهوم وسیع گفته می‌شود اگر میانگین فرآیند  $\mathbb{E}[x(t)] = m_x$  مستقل از زمان و تابع خودهمبستگی  $R(t + \tau, t) = \mathbb{E}[x(t)x(t + \tau)] = R_x(\tau)$  تنها تابعی از تأخیر  $\tau$  باشد.

برای فرآیند ایستاد با تابع خودهمبستگی  $R_x(\tau)$  چگالی طیف توان به صورت زیر محاسبه می‌شود.

$$S_x(f) = \mathcal{F}[R_x(\tau)] \quad (12)$$

یکی از موارد که به هنگام کار با سیستم‌های نامتغیر با زمان خطی مورد استفاده قرار می‌گیرد، طیف سیگنال خروجی از روی طیف سیگنال ورودی است. این رابطه در زیر آمده است.

$$S_y(f) = S_x(f)|H(f)|^2 \quad (13)$$



شکل ۵ عبور یک فرآیند از سیستم خطی نامتغیر با زمان

<sup>2</sup> In-Phase

<sup>3</sup> Quadrature



## شرح آزمایش



## آزمایش ۱-۳: معادل پایین‌گذر و میان‌گذر سیگنال

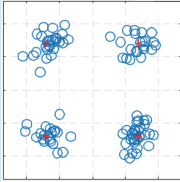
۱. تولید فرآیند میان‌گذر و پایین‌گذر: در صورتی که یک فرآیند تمام‌گذر از یک فیلتر پایین‌گذر عبور کند، یک فرآیند پایین‌گذر حاصل می‌شود. در این جا می‌خواهیم یک فرآیند پایین‌گذر را به معادل میان‌گذر تبدیل نماییم و مجدداً آن را به معادل پایین‌گذر تبدیل کنیم. کلیه‌ی طیف‌ها می‌بایست بر حسب dBm باشد و برچسب‌گذاری محور فرکانس به درستی انجام و بر حسب MHz بیان شود.

ا. تولید فرآیند پایین‌گذر: دنباله‌ی تصادفی مختلط با ۴۰۹۶ نمونه‌ی مستقل و با توزیع گاوسی مختلط با میانگین صفر و واریانس ۱ تولید نماییم. با استفاده از ابزار طراحی فیلتر نرم‌افزار MATLAB (دستور **filterDesigner**) را در پنجره‌ی دستور وارد نماییم و ضرایب فیلتر را به workspace ارسال نماییم (یک فیلتر پایین‌گذر FIR از نوع Equiripple و با پارامترهای فیلتری  $f_s = 28.8\text{MHz}$ ,  $f_{\text{pass}} = 1.4\text{MHz}$ ,  $f_{\text{stop}} = 2\text{MHz}$ ,  $A_{\text{pass}} = 0.5\text{dB}$  و  $A_{\text{stop}} = 70\text{dB}$  طراحی نماییم. پس از به دست آوردن ضرایب فیلتر با استفاده از عمل کانولوشن خروجی این فیلتر را برای ورودی تولید شده، به دست آورید. طیف ۴۰۹۶ نقطه‌ای فرآیند تمام‌گذر ورودی و فرآیند پایین‌گذر خروجی را به دست آورید و در یک شکل با دو زیرنمودار رسم نماییم. توان فرآیند را با استفاده از نمونه‌های زمانی قبل و بعد فیلتر شدن به دست آورده و ارتباط آن را با پهنای باند فیلتر به دست آورید. (راهنمایی: خروجی عمل کانولوشن تعداد نقاط بیشتری دارد و می‌بایست این تعداد نقاط را برابر با ۴۰۹۶ نقطه نمود. برای به دست آوردن پهنای باند فیلتر از دستور **noisebw** نرم‌افزار MATLAB استفاده نماید).

ب. تولید فرآیند میان‌گذر معادل یک فرآیند پایین‌گذر: حال با استفاده از مباحثی که در بخش تئوری مطرح شد، فرآیند پایین‌گذر تولید شده را به یک فرآیند میان‌گذر بر روی فرکانس 3.57MHz منتقل نماییم. توان هر سه فرآیند را مشابه قبل به دست آورید. آیا توان فرآیند معادل پایین‌گذر با میان‌گذر یکی است؟ علت را شرح دهید. حال طیف ۴۰۹۶ نقطه‌ای هر سه فرآیند را در یک شکل با سه زیرنمودار رسم نماییم.

ت. تولید فرآیند پایین‌گذر از یک سیگنال میان‌گذر: فرآیند تولید شده در بخش ب را دوباره به معادل پایین‌گذر خود تبدیل نموده و خروجی را از سه فیلتر پایین‌گذر مختلف با پهنای باند 0.7MHz، 1.4MHz و 2.8MHz عبور دهید. طیف فرآیند خروجی این سه فیلتر را با طیف بخش ا مقایسه نماییم و در هر مورد توان خروجی زمانی هر فیلتر را به دست آورید و علت تفاوت‌ها را شرح دهید. فیلترها را با استفاده از دستور **fir1** تولید نموده و مرتبه‌ی فیلتر را برابر با ۱۲۰ در نظر بگیرید. (فرکانس نمونه‌برداری همان  $f_s = 28.8\text{MHz}$  می‌باشد).





# آزمایش دوم

## معرفی مدولاسیون دیجیتال خطی

### اهداف آزمایش

سامانه‌های مخابرات دیجیتال بی سیم اطراف ما را فرا گرفته است و ارتباطات صدا و داده در سامانه‌های مخابرات سلولی، WiFi، Bluetooth و سایر استانداردها را فراهم می‌کند. در طی چندین آزمایش آینده بلوک‌های پایه‌ی مخابرات دیجیتال شامل الگوهای مدولاسیون، دریافت سمبل و بیت، عملکرد در حضور نویز، کدگذاری و کدگشایی تقاضایی، تبدیل فرکانس به بالا و پایین دیجیتال و ... ارائه می‌شود. در ادامه جزئیات طراحی گیرنده‌ی دیجیتال شامل همگام‌سازی فرکانس حامل و همگام‌سازی زمانی سمبل‌ها مورد بررسی قرار می‌گیرد. در این آزمایش با الگوهای مدولاسیون دیجیتال خطی و شکل‌دهی پالس آشنا می‌شویم.

در انتهای این آزمایش دانشجو می‌بایست:

- منظومه‌های مدولاسیون‌های خطی M-PAM، M-PSK و M-QAM را بشناسد.
- بتواند رشته‌های بیت را به سمبل‌های مدولاسیون‌های مختلف در حالت سمبل‌های مختلط و سمبل‌های I/Q تبدیل نماید.
- با مفهوم کدگذاری Gray آشنا شود.
- بتواند سمبل‌های دریافتی را به رشته‌های بیت تبدیل نماید.
- بتواند سیگنال را در یک کانال باندپایه ارسال نماید و اثر نویز را در تبدیل سمبل‌ها به رشته‌های بیت را بشناسد.
- با طراحی Correlator و فیلتر منطبق آشنا شود.
- بتواند گیرنده‌ی حداقل فاصله را طراحی نماید.

### ابزارهای مورد نیاز

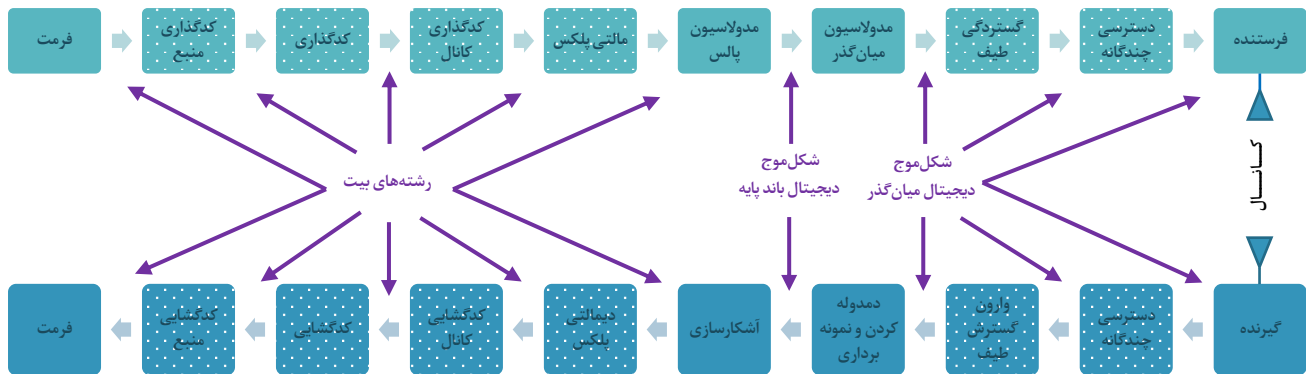
رایانه

نرم‌افزار MATLAB R2020b



## تئوری

شکل 1 شمای یک سامانه‌ی مخابرات دیجیتال متداول را به صورت کامل نشان می‌دهد. در این آزمایش و آزمایش بعد سعی داریم که بخش‌های ضروری این سامانه را برای مدولاسیون‌های خطی M-PAM، M-PSK و M-QAM مدل‌سازی کنیم. در ادامه این مدولاسیون‌ها را مرور خواهیم کرد.



شکل 1 شمای یک سامانه‌ی مخابرات دیجیتال (بخش‌های با پس‌زمینه‌ی نقطه‌دار اختیاری هستند)

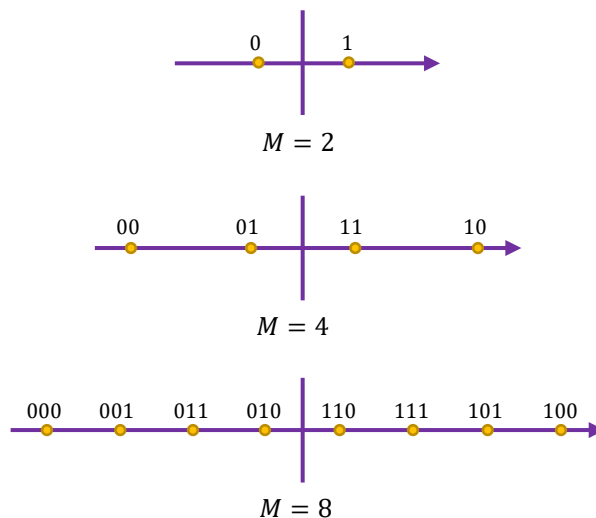
## مدولاسیون‌های خطی

### مدولاسیون M-PAM (Pulse Amplitude Modulation)

این مدولاسیون از جمله مدولاسیون‌های دیجیتال باند پایه می‌باشد. در این مدولاسیون اطلاعات در توان سیگنال ارسالی ذخیره می‌شود و آشکارسازی آن با تشخیص سطح سیگنال انجام می‌شود.

در این مدولاسیون سیگنال ارسالی به صورت  $s_m(t) = A_m p(t)$  است. پالس ارسالی  $p(t)$  می‌تواند به طور دلخواه انتخاب شود، اما از آنجا که پهنای باند سیگنال ارسالی به طور کامل به شکل پالس بستگی دارد، تعیین پالس مناسب اهمیت زیادی خواهد داشت.  $A_m$  دامنه پالس ارسالی را با توجه به پیغام تنظیم می‌کند. با فرض اینکه هر سیگنال ارسالی حاوی  $k$  بیت پیام باشد، می‌توان  $A_m$  ها را از مجموعه زیر انتخاب نمود.

$$A_m \in \{\pm 1, \pm 3, \dots, \pm(M-1)\}; \quad M = 2^k$$



شکل 2 منظومه‌ی سیگنالی مربوط به مدولاسیون M-PAM برای مقادیر مختلف M



## مدولاسیون PSK (Phase Shift Keying Modulation)

در این مدولاسیون، سیگنال ارسالی به صورت  $s_m(t) = g(t) \cos\left(2\pi f_0 t + \frac{2\pi}{M}(m-1)\right)$  می باشد که  $f_0$  فرکانس حامل خواهد بود. پالس  $g(t)$  می تواند به طور دلخواه انتخاب شود. در اینجا نیز همانند مدولاسیون PAM می توان با انتخاب درست این پالس، در پهنای باند صرفه جویی نمود. با فرض اینکه هر سیگنال ارسالی حاوی  $k$  بیت پیغام باشد، می توان  $m$  را از مجموعه  $\{1, 2, \dots, M\}$  انتخاب کرد. در این جا  $M = 2^k$  می باشد. با در نظر گرفتن سیگنال ارسالی به صورت فوق، خواهیم داشت.

با انتخاب  $\Phi_1(t) = \sqrt{\frac{2}{E_g}} g(t) \cos(2\pi f_0 t)$  و  $\Phi_2(t) = -\sqrt{\frac{2}{E_g}} g(t) \sin(2\pi f_0 t)$  به عنوان پایه های متعامد فضای سیگنال، خواهیم داشت.

$$s_m(t) = \Phi_1(t) \cos\left(\frac{2\pi}{M}(m-1)\right) \sqrt{\frac{E_g}{2}} + \Phi_2(t) \sin\left(\frac{2\pi}{M}(m-1)\right) \sqrt{\frac{E_g}{2}}$$

در رابطه ی فوق  $E_g$  انرژی پالس  $g(t)$  می باشد  $(\int |g(t)|^2 dt = E_g)$ . بدین ترتیب می توان معادل برداری سیگنال  $s_m(t)$  را به صورت زیر در نظر گرفت.

$$\bar{s}_m = \sqrt{\frac{E_g}{2}} \begin{bmatrix} \cos\left(\frac{2\pi}{M}(m-1)\right) \\ \sin\left(\frac{2\pi}{M}(m-1)\right) \end{bmatrix}$$

با توجه به  $\bar{s}_m$ ، فضای سیگنال این مدولاسیون دارای دو بعد در باند میانی می باشد. معادل باند پایه ی این سیگنال به صورت زیر خواهد بود:

$$s_{l,m}(t) = g(t) e^{j\frac{2\pi}{M}(m-1)}$$

با در نظر گرفتن  $\Phi_1(t) = \sqrt{\frac{2}{E_g}} g(t)$  به عنوان پایه متعامد فضای سیگنال، داریم:

$$s_{l,m}(t) = \sqrt{\frac{E_g}{2}} \Phi_1(t) e^{j\frac{2\pi}{M}(m-1)}$$

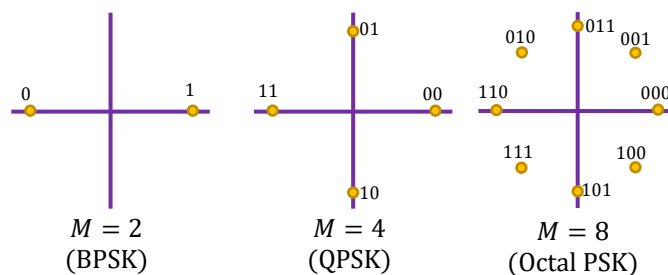
بنابراین در باند پایه، فضای سیگنال تنها یک بعد دارد. به بیان دیگر فضای سیگنال در این مدولاسیون دارای دو بعد حقیقی (باند میانی) یا یک بعد مختلط (باند پایه) خواهد بود. در باند پایه خواهیم داشت.

$$s_{l,m}(t) = g(t) \left( \cos\left(\frac{2\pi}{M}(m-1)\right) + j \sin\left(\frac{2\pi}{M}(m-1)\right) \right)$$

با متحد قرار دادن عبارت فوق با  $s_{l,m}(t) = s_i(t) + js_q(t)$ ، مؤلفه های هم فاز و غیرهم فاز به دست خواهند آمد.

$$\begin{cases} s_i(t) = g(t) \cos\left(\frac{2\pi}{M}(m-1)\right) \\ s_q(t) = g(t) \sin\left(\frac{2\pi}{M}(m-1)\right) \end{cases}$$

همان گونه که ملاحظه می شود، در این مدولاسیون، فاز سیگنال ارسالی، مشخص کننده ی پیغام می باشد.



شکل 3 منظومه ی مربوط به مدولاسیون MPSK برای مقادیر مختلف M

## مدولاسیون QAM (Quadrature Amplitude Modulation)

در این مدولاسیون، ابتدا سیگنال ارسالی را در باند پایه بررسی خواهیم کرد. سیگنال ارسالی مدولاسیون QAM در باند پایه به صورت زیر خواهد بود.

$$S_{l,m}(t) = (A_{m_i} + jA_{m_q})g(t)$$

$g(t)$  پالسی است که برای مدولاسیون انتخاب می شود و همانند دیگر مدولاسیون ها، بر پهنای باند مصرفی تأثیر می گذارد. بعد فضای این سیگنال در باند پایه ۱ می باشد. با در نظر گرفتن  $s_{l,m}(t) = s_i(t) + js_q(t)$ ، مؤلفه های in-phase و quadrature به صورت زیر خواهند بود.

$$\begin{cases} s_i(t) = g(t)A_{m_i} \\ s_q(t) = g(t)A_{m_q} \end{cases}$$

معادل باند میانی این سیگنال به صورت زیر به دست می آید.

$$S_m(t) = \text{Re}\{S_{l,m}(t)e^{j2\pi f_0 t}\} = A_{m_i}g(t)\cos(2\pi f_0 t) - A_{m_q}g(t)\sin(2\pi f_0 t)$$

با انتخاب  $\Phi_1(t) = \sqrt{\frac{2}{E_g}}g(t)\cos(2\pi f_0 t)$  و  $\Phi_2(t) = -\sqrt{\frac{2}{E_g}}g(t)\sin(2\pi f_0 t)$  به عنوان پایه های متعامد فضای سیگنال، خواهیم داشت:

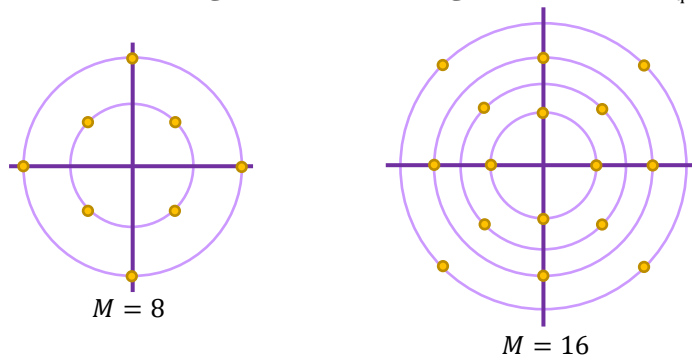
$$S_m(t) = A_{m_i}\sqrt{\frac{E_g}{2}}\Phi_1(t) + A_{m_q}\sqrt{\frac{E_g}{2}}\Phi_2(t)$$

در رابطه ی فوق نیز  $E_g$  انرژی پالس  $g(t)$  می باشد  $(\int |g(t)|^2 dt = E_g)$ .

بدین ترتیب می توان معادل برداری سیگنال  $s_m(t)$  را به صورت زیر در نظر گرفت:

$$\bar{s}_m = \sqrt{\frac{E_g}{2}} \begin{bmatrix} A_{m_i} \\ A_{m_q} \end{bmatrix}$$

به ازای مقادیر مختلف  $A_{m_i}$  و  $A_{m_q}$ ، فضای سیگنال می تواند شکل های گوناگونی مانند شکل 4 به خود بگیرد.

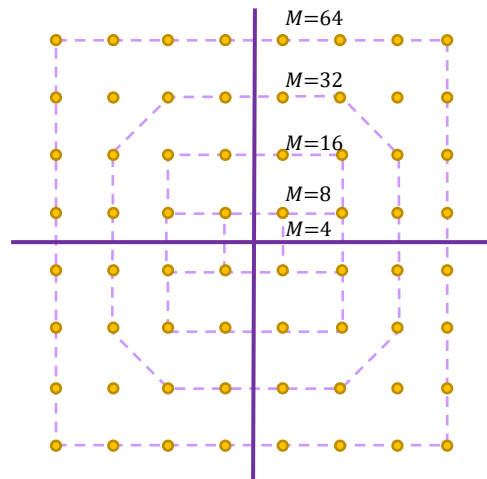


شکل 4 منظومه ی مدولاسیون M-QAM برای مقادیر مختلف M

حالت خاص فضای سیگنال به ازای  $A_{m_i}, A_{m_q} \in \{\pm 1, \pm 3, \dots, \pm(\sqrt{M}-1)\}$  تشکیل می شود که به QAM مستطیلی معروف

است. منظومه ی این مدولاسیون در شکل 5 نشان داده شده است.

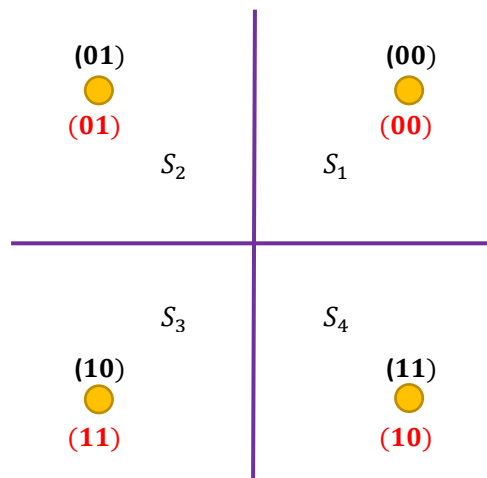
از این به بعد، هر جا که درباره مدولاسیون QAM صحبت می شود منظور QAM مستطیلی خواهد بود.



شکل 5 منظومه‌ی مدولاسیون M-QAM مستطیلی برای مقادیر مختلف M

### کدگذاری Gray

بسته به مدولاسیونی که برای ارسال انتخاب می‌شود، هر  $k$  بیت از رشته بیت تولیدی به یک سمبل مشخص تبدیل می‌شود. به عنوان مثال شکل 6، نگاشت طبیعی (اعداد مشکی) و گری (اعداد قرمز) را در مدولاسیون 4QAM نشان می‌دهد. همان‌طور که در شکل مشاهده می‌شود، با استفاده از کدگذاری گری، اگر تحت تأثیر نویز، یک سمبل با سمبل مجاور خود جابه‌جا شود، فقط یک بیت به اشتباه کدگذاری می‌شود؛ اما در صورت استفاده از کدگذاری طبیعی امکان دارد که هر دو بیت به اشتباه کدگذاری شود. از این رو معمولاً از کدگذاری گری استفاده کنیم.



شکل 6 نگاشت طبیعی (عددهای مشکی) و نگاشت گری (عددهای قرمز)

در کد گری، هر دو دنباله‌ی متوالی فقط در یک بیت با یکدیگر اختلاف دارند. برای تولید دنباله‌ی  $n$  بیتی این کد به روش زیر عمل

می‌کنیم.

| $n = 1$                                                   | $n = 2$                                                                                           | $n = 3$                                                                                           | $n = 4$                                                                                            | ... |
|-----------------------------------------------------------|---------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|-----|
| $A_1 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}_{2 \times 1}$ | $A_2 = \begin{bmatrix} 0_{2 \times 1} & A_1 \\ 1_{2 \times 1} & A_1^B \end{bmatrix}_{4 \times 2}$ | $A_3 = \begin{bmatrix} 0_{4 \times 1} & A_2 \\ 1_{4 \times 1} & A_2^B \end{bmatrix}_{8 \times 3}$ | $A_4 = \begin{bmatrix} 0_{8 \times 1} & A_3 \\ 1_{8 \times 1} & A_3^B \end{bmatrix}_{16 \times 4}$ | ... |

در روند ذکر شده، ماتریس  $A_i^B$  از معکوس کردن هر یک از ستون‌های  $A_i$  به دست می‌آید. به عبارتی عملگر ماتریسی  $B$  به صورتی که در ادامه آمده است، عمل می‌نماید.

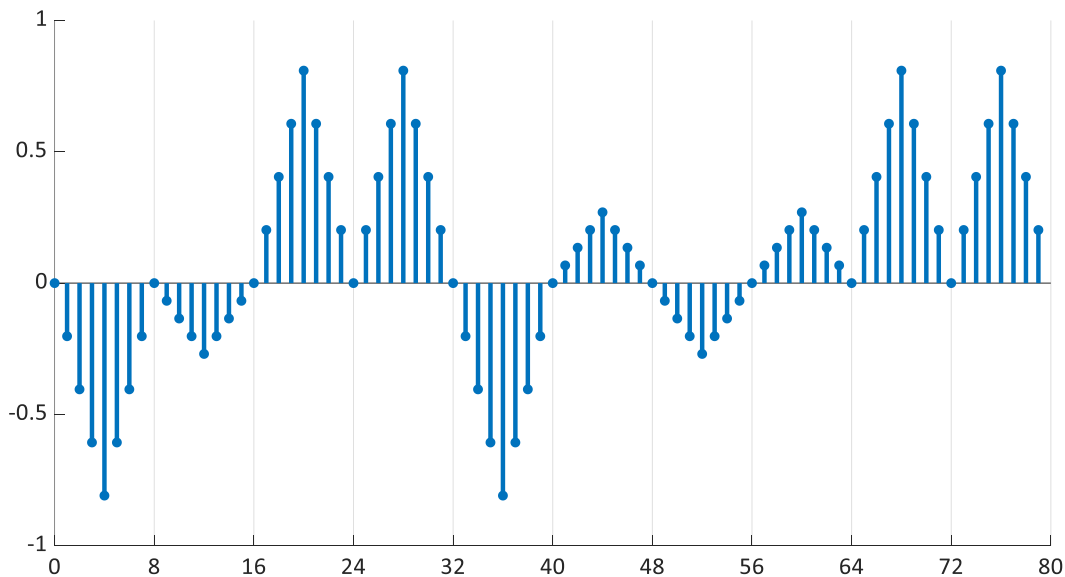
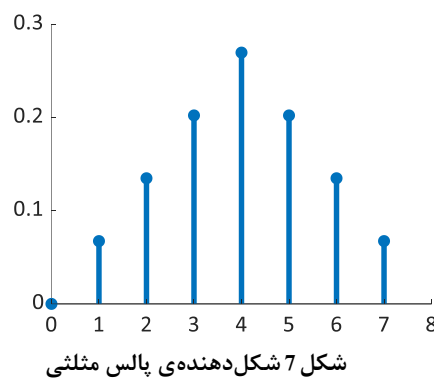
$$A_i = \begin{bmatrix} a_{1,1} & a_{1,2} & \dots & a_{1,i} \\ a_{2,1} & a_{2,2} & \dots & a_{2,i} \\ \vdots & \vdots & \ddots & \vdots \\ a_{2^{i-1},1} & a_{2^{i-1},2} & \dots & a_{2^{i-1},i} \end{bmatrix} \Rightarrow A_i^B = \begin{bmatrix} a_{2^i,1} & a_{2^i,2} & \dots & a_{2^i,i} \\ a_{2^{i-1},1} & a_{2^{i-1},2} & \dots & a_{2^{i-1},i} \\ \vdots & \vdots & \ddots & \vdots \\ a_{1,1} & a_{1,2} & \dots & a_{1,i} \end{bmatrix}$$

### اعمال شکل دهی پالس

پس از انتخاب سمبل‌ها لازم است که از شکل دهی پالس برای ارسال سمبل‌ها استفاده کنیم. سیگنال شکل دهنده‌ی پالس را با  $p(t)$  نشان می‌دهیم. شکل 7،  $p(t)$  مثلثی را نشان می‌دهد.

فرض کنید که می‌خواهیم یک دنباله‌ی ۱۰ تایی از سمبل‌های 4PAM را توسط  $p(t)$  نشان داده در شکل 7 ارسال کنیم. در صورتی که دنباله‌ی سمبل‌ها به صورت  $s = -3, -1, 3, 3, -3, 1, -1, 1, 3, 3$  باشد، سیگنال ارسالی به صورت شکل 8 خواهد بود.

برای تشکیل سیگنال ارسالی، دو روش ضرب کرونیگر<sup>۴</sup> و استفاده از کانولوشن وجود دارد که در ادامه به توضیحاتی در مورد آن‌ها می‌پردازیم.



### تولید سیگنال ارسالی با استفاده از ضرب کرونیگر

ضرب کرونیگر دو ماتریس  $A$  و  $B$  را که با  $A \oplus B$  نشان می‌دهیم، به صورت زیر تعریف می‌شود.

<sup>۴</sup> Kronecker Product

$$\mathbf{A}_{m \times n} \oplus \mathbf{B}_{p \times q} = \begin{bmatrix} A_{11}\mathbf{B} & \dots & A_{1n}\mathbf{B} \\ \vdots & \ddots & \vdots \\ A_{m1}\mathbf{B} & \dots & A_{mn}\mathbf{B} \end{bmatrix}$$

در صورتی که  $\mathbf{A}$  و  $\mathbf{B}$  هر دو بردارهای سطری باشند، حاصل این عملگر به صورت زیر خواهد بود.

$$\mathbf{A}_{1 \times n} \oplus \mathbf{B}_{1 \times q} = [A_1\mathbf{B} \quad A_2\mathbf{B} \quad \dots \quad A_n\mathbf{B}]$$

بنابراین در اینجا می‌توان سیگنال ارسالی شکل 8 را با استفاده از ضرب کرونیگر تولید کرد.

### تولید سیگنال ارسالی با استفاده از کانولوشن

دنباله‌ی سمبل‌ها را می‌توان به صورت  $s = \sum_{i=0} s_i \delta[n - i]$  نشان داد.  $s$  را می‌توان به صورت زیر Zero Pad نمود.

$$s' = \sum_{i=0} s_i \delta[n - iL], \quad L = \text{length}(p)$$

حاصل کانولوشن سیگنال  $s'$  و  $p$  به صورت زیر خواهد بود که همان نتیجه‌ی مطلوب را به دست می‌دهد.

$$s' * p = \left( \sum_{i=0} s_i \delta[n - iL] \right) * p = \sum_{i=0} s_i p[n - iL]$$

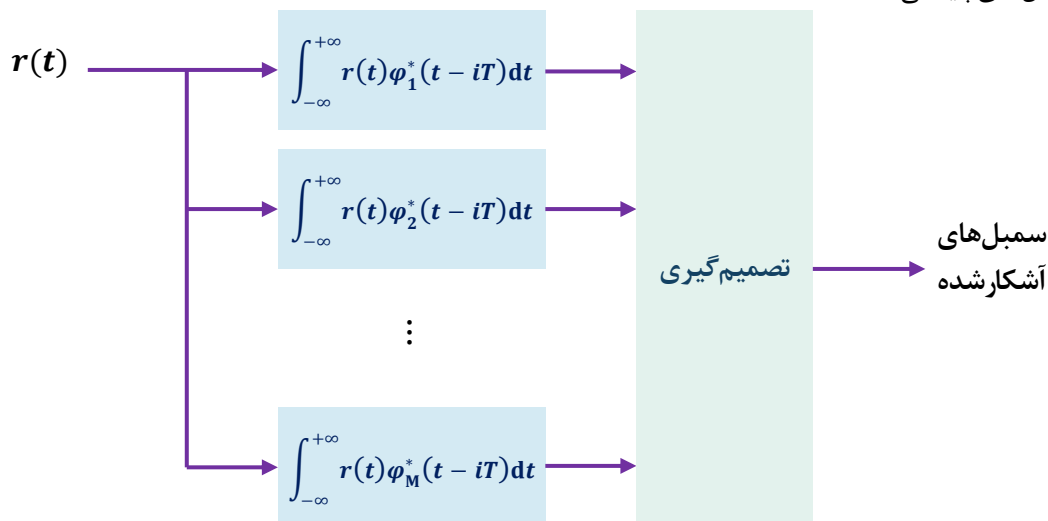
نکته: در صورتی که پالس‌های ارسالی با یکدیگر تداخل داشته باشند، روش ضرب کرونیگر سیگنال صحیح را تولید نمی‌کند.

### تشخیص تصویر سیگنال دریافتی در راستای سیگنال‌های پایه

از آنجا که سمبل‌های ارسالی توسط سیگنال‌های پایه ارسال شده‌اند، می‌بایست در گیرنده، قدرت سیگنال در راستای هر یک از پایه‌ها به دست آید تا سیگنال دریافتی را آشکار کنیم. این کار با استفاده از Correlator و یا فیلتر منطبق<sup>۵</sup> انجام می‌شود.

#### Correlator

شکل 9 شمای کلی یک گیرنده را با استفاده از Correlator (بلوک‌های آبی رنگ) نشان می‌دهد. خروجی هر یک از بلوک‌های خاکستری، قدرت سیگنال را در راستای یکی از سیگنال‌های پایه نشان می‌دهد. این مقادیر پس از ورود به بلوک تصمیم‌گیری، با منظومه‌ی سیگنالی مدولاسیون مقایسه می‌شوند و نزدیک‌ترین سمبل به آن به عنوان سمبل ارسالی انتخاب می‌شود. در بلوک‌های مشخص شده،  $T$  طول زمانی سیگنال‌های پایه می‌باشد.



شکل 9 استفاده از Correlator برای اندازه‌گیری قدرت سیگنال در راستای هر سیگنال پایه

### فیلتر منطبق

اگر بنا باشد به جای استفاده از انتگرال گیر، از فیلتری استفاده شود که خروجی آن شبیه Correlator است، به صورت زیر عمل

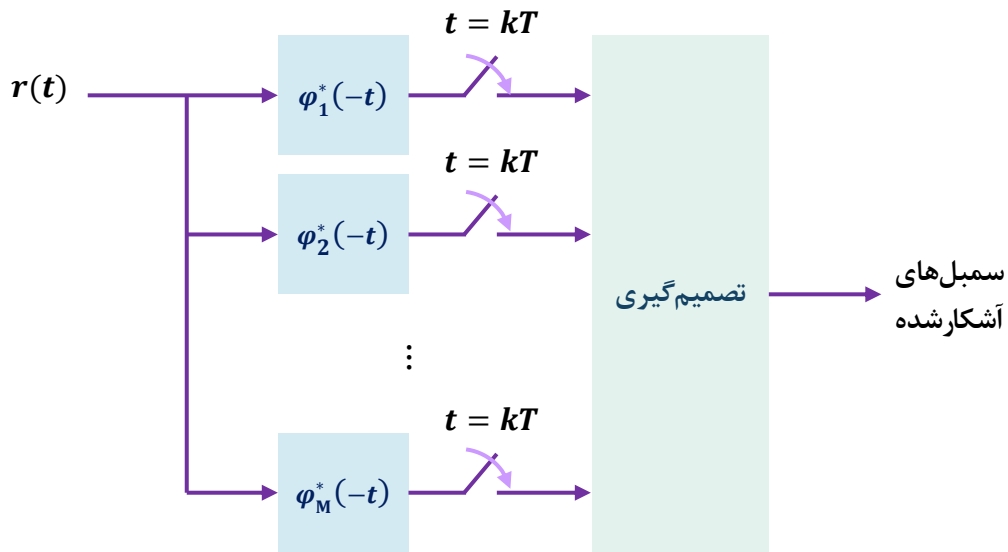
می‌شود.

<sup>5</sup> Matched Filter



$$\int_{-\infty}^{\infty} r(t) \cdot \phi_m^*(t - iT) dt = \int_{-\infty}^{\infty} r(\tau) \cdot \phi_m^*(\tau - iT) d\tau = r(t) * \phi_m^*(-t)|_{t=iT}$$

با توجه به رابطه‌ی فوق می‌توان به جای استفاده از انتگرال گیر، فیلتر  $\phi_m^*(-t)$  را بر سر راه سیگنال دریافتی قرار داد و در نقاط  $iT$  از آن نمونه‌گیری کرد. شکل 10، ساختار کلی یک گیرنده را با استفاده از Matched Filter نشان می‌دهد.



شکل 10 استفاده از فیلتر منطبق برای اندازه‌گیری قدرت سیگنال در راستای هر سیگنال پایه

### گیرنده‌ی حداقل فاصله

به گیرنده‌ای که سمبل دریافتی را با منظومه‌ی سیگنالی یک مدولاسیون مقایسه و نزدیکترین سمبل به سیگنال دریافتی را از میان سمبل‌های موجود در منظومه‌ی سیگنالی انتخاب کند، گیرنده‌ی حداقل فاصله گفته می‌شود.

## شرح آزمایش

### آزمایش ۱-۲: پیاده‌سازی فرستنده مدولاسیون PAM

در این آزمایش بنا داریم تعدادی بسته‌ی داده را برای ارسال از طریق مدولاسیون 4PAM آماده نماییم. از آن‌جا که در آزمایش‌های آینده، بناسست این فرستنده با استفاده از رادیو نرم‌افزار ADALM-PLUTO پیاده‌سازی شود، بنابراین می‌بایست با استفاده از پارامترها و ملاحظات عملی، این کار انجام شود.

پارامترهایی که در این پیاده‌سازی اهمیت دارد شامل نرخ ارسال بیت (**bitRate**)، ضریب افزایش نرخ نمونه‌برداری سمبل‌ها یا تعداد نمونه‌های هر سمبل (**nSymbolSamples**)، تعداد سمبل‌های ارسالی در هر بسته‌ی داده (**nPacketSymbols**) و مدت زمان ارسال داده (**dataTransferDuration**) می‌باشد. در این آزمایش مقدار پارامترهای مورد نیاز به صورت جدول 1 می‌باشد.

جدول 1 پارامترهای آزمایش ۱-۴

| پارامتر | nSymbolSamples | nPacketSymbols |
|---------|----------------|----------------|
| مقدار   | ۸              | ۱۰             |

۱. تولید بیت: با استفاده از تابع **bitGenerator** تعدادی بیت ۰ و ۱ را متناسب با طول بسته‌ی داده و با در نظر گرفتن پارامترهای مدولاسیون 4PAM تولید نموده و آن را درون ماتریس **txBit** ذخیره نمایید. ماتریس **txBit** یک ماتریس با اندازه‌ی  $nPacketSymbols \times k$  (۱۰×۲) می‌باشد.

۲. نگاشت بیت به سمبل:

گام ۱. تولید دنباله‌ی کد گری به صورت باینری: تابعی بنویسید که آرگومان ورودی آن تعداد بیت‌های دنباله باشد و خروجی آن کد گری مربوط به این دنباله باشد. سطر اول این تابع می‌بایست به صورت زیر باشد.

```
function [grayMatrix] = grayMatrixGenerator(k)
```

گام ۲. کدگذاری گری بیت‌های تولیدی: با استفاده از تابع **grayMatrixGenerator**، ماتریس مربوط به کدگذاری گری باینری مدولاسیون 4PAM را تولید کنید و آن را در ماتریس **grayMatrix** ذخیره کنید. بردار جدیدی متناظر با ماتریس **txBit** و با نام **symbolIndex** تولید نمایید و نگاشتی یک‌به‌یک بین سطرهای ماتریس **txBit** و ماتریس **grayMatrix** برقرار کنید. به عبارتی سطر  $\lambda$ ام بردار **symbolIndex**، شماره‌ی سطری از **grayMatrix** را نشان می‌دهد که برابر با سطر  $\lambda$ ام ماتریس **txBit** باشد.

گام ۳. تولید سمبل‌های ارسالی: با استفاده از تابع **constellation**، تمامی سمبل‌های ارسالی یک مدولاسیون 4PAM را تولید کرده و سمبل‌های متناظر با هر سطر بردار **symbolIndex** را تولید کنید. این سمبل‌ها را در بردار **symbolArray** ذخیره نمایید.

۳. شکل‌دهی پالس ارسالی: همان‌طور که در بخش تئوری بیان گردید، سیگنال ارسالی را می‌توان با دو روش ضرب کرونیگر و کانولوشن تولید و پالس ارسالی را شکل‌دهی نمود. تابعی بنویسید که بردار شماره‌ی سمبل‌های تولیدشده (**symbolIndex**)، مدولاسیون (**modulation**)، مرتبه‌ی مدولاسیون (**M**)، نرخ نمونه‌برداری (**fs**)، تعداد نمونه‌های هر سمبل (**nSymbolSamples**)، نام تابع شکل‌دهنده‌ی پالس (**pulseName**) و روش تولید شکل‌دهی پالس (**pulseShapingMode**) را دریافت کند و نمونه‌های سیگنال ارسالی متناظر با یک بسته و منظومه‌ی سیگنالی را تولید کند. سطر اول این تابع می‌بایست به صورت زیر باشد.

```
function [txSamples, constellation] = pulseModulation(symbolIndex, modulation, M, fs, nSymbolSamples, pulseName, pulseShapingMode, varargin)
```

توجه داشته باشید که گام ۳ سوال قبل را مجدد در قالب یک تابع منسجم انجام داده‌ایم. در این‌جا **pulseShapingMode** روش شکل‌دهی پالس می‌باشد که یکی از دو حالت 'conv' و 'kron' را اختیار می‌کند.

با استفاده از این تابع نمونه‌های ارسالی مربوط به یک بسته‌ی داده را آماده‌ی ارسال کرده و حاصل را در بردار **txSamples** ذخیره نمایید. تابع شکل‌دهی پالس در این آزمایش پالس مثلثی است. اثر شکل پالس‌ها در آزمایش بعد مورد بررسی قرار می‌گیرد.

(راهنمایی ۱: ضرب کرونیگر در برنامه‌ی MATLAB با استفاده از دستور **kron** انجام می‌شود که شیوه‌ی استفاده از آن به صورت **txSamples = kron(symbolArray, p)** می‌باشد.)

(راهنمایی ۲: برای انجام عمل Zero Padding از تابع **upsample** نرم‌افزار MATLAB استفاده کنید. دقت نمایید که تابع شما نباید بعد از آخرین سمبل صفر اضافه نماید. حال سیگنال ارسالی را توسط عملگر کانولوشن و به صورت **txSamples = conv(symbolArrayZeroPad, p)** تولید نمایید.)

## آزمایش ۲-۲: پیاده‌سازی گیرنده‌ی مدولاسیون PAM

در ادامه‌ی می‌خواهیم گیرنده‌ی یک مدولاسیون PAM را پیاده‌سازی نماییم.

۱. تشخیص قدرت سیگنال دریافتی در راستای سیگنال‌های پایه: تابعی بنویسید که دو بردار **rxSamples** و **p** را دریافت کند و قدرت سیگنال را در راستای **p** محاسبه کند. این تابع باید بتواند قدرت سیگنال را در راستای **p**، با استفاده از هر دو روش correlator و فیلتر منطبق محاسبه کند. سطر اول این تابع می‌بایست به صورت زیر باشد.

```
function [rxSymbols] = symbolDetection(rxSamples, p, nSymbolSamples, rxMode)
```

در این تابع **rxMode** می‌تواند یکی از دو حالت 'correlator' و یا 'matched\_filter' را اختیار کند.

۲. گیرنده‌ی حداقل فاصله: تابعی بنویسید که منظومه‌ی سیگنالی (**constellation**) و بردار سمبل‌های دریافتی (**rxSymbols**) را به عنوان ورودی بگیرد و در خروجی، برداری به طول **r** تولید کند. المان  $i$ ام بردار سمبل‌های آشکارشده‌ی **detectedSymbols**، نزدیکترین مقدار **constellation** به المان  $i$ ام بردار **rxSymbols** خواهد بود. سطر اول این تابع به صورت زیر می‌باشد.

```
function [detectedSymbols] = minDistanceDetector(rxSymbols, constellation);
```

۳. دمدولاسیون یا آشکارسازی پالس: نمونه‌های ارسالی شده (**txSamples**) را به عنوان نمونه‌های دریافتی درون بردار **rxSamples** قرار دهید. در این جا می‌خواهیم تابعی بنویسیم که دو عمل قبل را به صورت منسجم در قالب یک تابع انجام دهد. ورودی‌های این تابع نمونه‌های سیگنال دریافتی (**rxSamples**)، نام مدولاسیون (**modulation**)، مرتبه‌ی مدولاسیون (**M**)، نرخ نمونه‌برداری (**fs**)، تعداد نمونه‌های هر سمبل (**nSymbolSamples**)، نام تابع شکل‌دهنده‌ی پالس (**pulseName**) و روش تشخیص قدرت سیگنال دریافتی در راستای سیگنال‌های پایه (**rxMode**) را دریافت کند و خروجی آن سمبل‌های دریافتی (**rxSymbols**) و اندیس سمبل‌های آشکارشده (**detectedSymbolsIndex**) باشد.

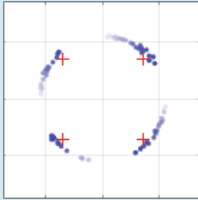
```
function [detectedSymbolsIndex, rxSymbols] = pulseDemodulation(rxSamples, modulation, M, fs, nSymbolSamples, pulseName, rxMode, varargin)
```

۴. تبدیل از کدگذاری گری به کدگذاری باینری: به کمک بردار **detectedSymbolsIndex** و ماتریس **grayMatrix** اندیس سمبل‌های آشکارشده‌ی با کدگذاری باینری را به دست آورید و درون متغیر **detectedBits** قرار دهید. برای این منظور متناظر با محتوای هر سطر **detectedSymbolsIndex** سطری متناظر با آن محتوا از **grayMatrix** انتخاب می‌شود.



- [1] J. G. Proakis and M. Salehi, *Digital Communications*. Boston: McGraw-Hill, 2008.
- [2] M. Rice, *Digital Communications: A Discrete-Time Approach*. Upper Saddle River, NJ: Pearson Education, 2009.
- [3] R. W. Stewart, K. Barlee, L. Crockett, and D. Atkinson, *Software Defined Radio using MATLAB & Simulink and the RTL-SDR*. 2015.





# آزمایش سوم

## کاوش در مدولاسیون دیجیتال خطی

### اهداف آزمایش

سامانه‌های مخابرات دیجیتال بی‌سیم اطراف ما را فرا گرفته است و ارتباطات صدا و داده در سامانه‌های مخابرات سلولی، WiFi، Bluetooth و سایر استانداردها را فراهم می‌کند. در طی چندین آزمایش آینده بلوک‌های پایه‌ی مخابرات دیجیتال شامل الگوهای مدولاسیون، دریافت سمبل و بیت، عملکرد در حضور نویز، کدگذاری و کدگشایی تقاضای، تبدیل فرکانس به بالا و پایین دیجیتال و ... ارایه می‌شود. در این آزمایش در ادامه‌ی آزمایش قبل با الگوهای مدولاسیون دیجیتال خطی و شکل‌دهی پالس آشنا می‌شویم و به کاوش بر روی پارامترهای مختلف این مدولاسیون‌ها می‌پردازیم. هم‌چنین تأثیراتی که کانال می‌تواند بر روی فرآیند آشکارسازی بگذارد نیز مورد بررسی قرار می‌گیرد.

در انتهای این آزمایش دانشجو می‌بایست:

- منظومه‌های مدولاسیون‌های خطی M-PAM، M-PSK و M-QAM را بشناسد.
- بتواند رشته‌های بیت را به سمبل‌های مدولاسیون‌های مختلف در حالت سمبل‌های مختلط و سمبل‌های I/Q تبدیل نماید.
- بر روی کدگذاری Gray برای مدولاسیون‌های مختلف مسلط باشد.
- با علت استفاده از شکل‌دهی پالس در سمبل‌های ارسالی آشنا شود.
- با اثراتی که شکل‌دهی پالس بر روی پهنای باند طیف ارسالی آشنا شود.
- بتواند سیگنال را در یک کانال باندپایه ارسال نماید و اثر نویز را در تبدیل سمبل‌ها به رشته‌های بیت را بشناسد.
- بتواند به کانال باندپایه، تأخیر و اختلاف فاز اضافه نماید و با اثر این عوامل بر روی منظومه‌ی سیگنالی و خطای آشکارسازی آشنا شود.
- با طراحی Correlator و فیلتر منطبق آشنا شود.
- بتواند سمبل‌های دریافتی مدولاسیون‌های مختلف را به رشته‌های بیت تبدیل نماید.
- بتواند گیرنده‌ی حداقل فاصله را طراحی نماید.

### ابزارهای مورد نیاز

- رایانه
- نرم‌افزار MATLAB R2020b
- رادیو نرم‌افزار ADALM-PLUTO
- کابل کوکسیال و آنتن



## تئوری

## تعیین نسبت انرژی متوسط بیت به چگالی نویز

به منظور بررسی رفتار و عملکرد مدولاسیون های دیجیتال خطی در عبور از یک کانال با نویز سفید جمع شونده می بایست پارامتر نسبت انرژی متوسط بیت به چگالی نویز ( $E_b/N_0$ ) به درستی تعریف شود. انرژی متوسط بیت را می توان بر اساس تعداد بیت های معرف یک سمبل و انرژی متوسط سمبل یک مدولاسیون دیجیتال تعیین نمود. با داشتن نسبت  $\frac{E_b}{N_0}$  می توان  $N_0$  را محاسبه نمود. البته ذکر این نکته حایز اهمیت است که در سامانه های عملی، ابتدا مقدار  $N_0$  که به پارامترهای گیرنده ارتباط دارد محاسبه شده و سپس توان فرستنده برای رسیدن به انرژی متوسط بیت مورد نظر تعیین می گردد. از آن جا که برای محاسبه ی عملکرد یک مدولاسیون در محیط AWGN تنها نسبت  $E_b/N_0$  اهمیت دارد، در شبیه سازی استفاده از هر یک از دو رویکرد فوق چندان تفاوتی ندارد.



## شرح آزمایش

## آزمایش ۱-۳: مقداردهی های اولیه

در این آزمایش بنا داریم تعدادی بسته ی داده را برای ارسال از طریق مدولاسیون های خطی آماده نماییم. از آن جا که در این آزمایش بناسست این فرستنده با استفاده از رادیو نرم افزار ADALM-PLUTO پیاده سازی شود، می بایست با استفاده از پارامترها و ملاحظات عملی، این کار انجام شود.

پارامترهایی که در این پیاده سازی اهمیت دارد شامل نرخ نمونه برداری ( $f_s$ )، ضریب افزایش نرخ نمونه برداری سمبل ها یا تعداد نمونه های هر سمبل ( $nSymbolSamples$ )، تعداد سمبل های ارسالی در هر بسته ی داده ( $nPacketSymbols$ ) و مدت زمان ارسال داده ( $dataTransferDuration$ ) می باشد. در این آزمایش مقدار پارامترهای مورد نیاز به صورت جدول 1 می باشد.

جدول 2 پارامترهای آزمایش ۵

| پارامتر | $f_s$ | $nSymbolSamples$ | $nPacketSymbols$                     | $dataTransferDuration$ |
|---------|-------|------------------|--------------------------------------|------------------------|
| مقدار   | 10MHz | ۸                | شبیه سازی: ۱۰۰۰۰۰<br>سخت افزار: ۱۰۰۰ | ۱۰۰ ثانیه              |

دقت نظر داشته باشید که در هر آزمایش تنها می خواهیم برنامه هایی که در آزمایش قبل نوشته شده است را تکمیل نماییم. هم چنین ابتدای برنامه نویسی  $nPacketSymbols$  را مقدار کوچکی مانند ۱۰ قرار دهید و پس از اطمینان از برنامه مقدار آن را افزایش دهید.

- تولید M-File مربوط به مقدار دهی اولیه: در آزمایش ها می خواهیم، مدولاسیون های مختلف، عوامل غیریافته آل کانال، استفاده یا عدم استفاده از سخت افزار، شکل پالس های مختلف و ... را مورد بررسی قرار دهیم. از این رو بهتر است برنامه به صورت یک کل نوشته شود تا با تغییر چند پارامتر در فایل مقداردهی اولیه، تغییرات مد نظر در کل برنامه اعمال شود. در ادامه قالب کلی این پارامترها آورده شده است. فایل `dclInitialize.m` را ایجاد نموده و این موارد را در آن وارد نمایید. سپس می بایست این فایل را از برنامه ی اصلی خود فراخوانی نمایید.

```
%% Simulation Parameters
isHardware = 0;
dataTransferDuration = 100;
```

```
%% Receiver Parameters
fs = 10e6;
61.44e6 Hz)
ts = 1/fs;
```

```
% BasbitAverageEnergyand Sampling Rate (65105 to
```

```
% BasbitAverageEnergyand Sampling Time
```



```

nPacketSymbols = 1e6; % Number of Symbol in Each Packet
rxReceiveMode = 0; % Receiver Detection Algorithm
equalizerMode = 0; % Compensate Mode (0: No Compensation, 1:
Amplitude Compensation, 2: Phase Compensation, 3: Compensation)
%% Modulation Parameters
modulation = 'psk'; % Modulation Name ('psk', 'pam', 'qam', 'fsk')
k = 2; % Bit Per Symbol
M = 2^k; % Modulation Order

nSymbolSamples = 8; % Sample Per Symbol
Ts = nSymbolSamples*ts; % Symbol Time

isGray = 1; % Gray Code Usage Flag
detectionMode = 'coherent'; % Modulation Detection Option ('coherent',
'noncoherent')

% Pulse Shape Parameters
pulseName = 'triangular'; % Name of Pulse Shaping Function
beta = 0.99; % Parameter for RC, RRC and Gaussian Pulse Shape
spanInSymbol = 1; % Truncation Length for RC, RRC and Gaussian
Pulse Shape (Multiple of Symbol Time)

% Header Option
isHeader = 0; % Flag For Having Packets with Header

% SNR Bound for BER Plots
snrMin = 0; % Minimum SNR (dB)
snrMax = 10; % Maximum SNR (dB)
snrStep = 1; % SNR Step (dB)
snrDb = snrMin:snrStep:snrMax; % SNR Vector (dB)

%% Channel Parameters
channelDelayInSample = 0; % Channel Delay in Sample
channelPhaseOffset = 0 * pi/180; % Channel Phase Offset
channelFrequencyOffset = 0; % Channel Frequency Offset

%% Hardware Parameters
% Transmitter Parameters
txFc = 2400e6; % Set Transmitter Center Frequency (AD9363: 325-
3800MHz) (AD9364: 70-6000MHz)
txGain = 0; % Set Transmitter Attenuation As a Negative Gain
(-89.75 to 0 dB)
txAddress = 'usb:0'; % Set Transmitter Identification Number

% Receiver Parameters
rxFc = 2400e6; % Set Receiver Center Frequency (AD9363: 325-
3800MHz) (AD9364: 70-6000MHz)
rxGain = 20; % Set Receiver Gain (-4dB to 71dB)
rxAddress = 'usb:0'; % Set Receiver Identification Number

% Initialize ADALM-PLUTO
if isHardware
    dev = sdrdev('Pluto'); % Create Radio Object for ADALM-PLUTO
    setupSession(dev)
    configurePlutoRadio('AD9364'); % Configure ADALM-PLUTO Radio Firmware
end

```



## آزمایش ۲-۳: پیاده‌سازی فرستنده مدولاسیون

۱. تولید بیت: با استفاده از تابع `bitGenerator` تعدادی بیت ۰ و ۱ را متناسب با طول بسته‌ی داده و با در نظر گرفتن پارامترهای مدولاسیون 4PSK تولید نموده و آن را درون ماتریس `txBit` ذخیره نمایید. ماتریس `txBit` یک ماتریس با اندازه‌ی  $nPacketSymbols \times k$  می‌باشد.
۲. نگاشت بیت به سمبل:
  - گام ۱. کدگذاری گری بیت‌های تولیدی: با استفاده از تابع `grayMatrixGenerator`، ماتریس مربوط به کدگذاری گری مدولاسیون 4PSK را تولید کنید و آن را در ماتریس `grayMatrix` ذخیره کنید. بردار جدیدی متناظر با ماتریس `txBit` و با نام `symbolIndex` تولید نمایید و نگاشتی یک‌به‌یک بین سطرهای ماتریس `txBit` و ماتریس `grayMatrix` برقرار کنید. به عبارتی سطر  $i$ ام بردار `symbolIndex`، شماره‌ی سطر  $i$  از `grayMatrix` را نشان می‌دهد که برابر با سطر  $i$ ام ماتریس `txBit` باشد. (راهنمایی: بهتر است سطرهای ماتریس `grayMatrix` را به یک عدد صحیح تبدیل نمایید).
  - گام ۲. کدگذاری طبیعی بیت‌های تولیدی: با استفاده از فلگ `isGray`، قابلیت‌ی را به برنامه‌ی خود اضافه نمایید که بتوان کدگذاری گری را اعمال نمود.
  - گام ۳. تولید سمبل‌های ارسالی: با استفاده از تابع `constellation`، تمامی سمبل‌های ارسالی مدولاسیون 4PSK را تولید کرده و سمبل‌های متناظر با هر سطر بردار `symbolIndex` را تولید کنید. این سمبل‌ها را در بردار `symbolArray` ذخیره نمایید.
۳. شکل‌دهی پالس ارسالی: با استفاده از تابع `pulseModulation` نمونه‌های ارسالی مربوط به یک بسته‌ی داده را آماده‌ی ارسال کرده و حاصل را در بردار `txSamples` ذخیره نمایید. تابع شکل‌دهی پالس را فعلاً مثلاً در نظر بگیرید.

## آزمایش ۳-۳: مدل‌سازی کانال

۱. افزودن تأخیر در کانال: برای شبیه‌سازی تأخیر در کانال به اندازه‌ی پارامتر `channelDelayInSample` به ابتدای بردار `txSamples` صفر اضافه نمایید. حاصل را درون بردار `txSamplesDelayed` قرار دهید. دقت نمایید که طول بردار افزایش می‌باید و می‌بایست در ادامه‌ی برنامه اثر این افزایش لحاظ شود. در این جا فعلاً مقدار تأخیر برابر با صفر قرار دهید.
۲. اعمال اختلاف فاز در کانال: برای اعمال اختلاف فاز در کانال می‌بایست بردار `txSamplesDelayed` را در  $\exp(i \cdot \text{channelPhaseOffset})$  ضرب شود. حاصل را در بردار `rxSamples` قرار دهید. در این جا نیز آفست فاز را برابر با صفر قرار دهید.
۳. شبیه‌سازی کانال با نویز سفید گاوسی
  - گام ۱. تعیین واریانس نویز بر اساس نسبت سیگنال به نویز ( $E_b/N_0$ ): ابتدا بر اساس خروجی تابع `constellation`، مقدار متوسط انرژی سمبل را به دست آورید (`symbolAverageEnergy`). حال با استفاده از متوسط انرژی سمبل، مقدار انرژی متوسط بیت را به دست آورده و برابر متغیر `bitAverageEnergy` قرار دهید. مقدار  $E_b/N_0$  را برابر با 3dB در نظر بگیرید (به عبارتی پارامترهای `snrMin` و `snrMax` برابر 3 تنظیم می‌شوند). سپس واریانس نویز را بر اساس نسبت  $E_b/N_0$  به دست آورید و آن را درون متغیر `noiseVariance` قرار دهید.
  - گام ۲. افزودن نویز به سیگنال: با استفاده از تابع `randn` یک بردار نویز مختلط با واریانس `noiseVariance` و ابعاد برابر با `txSamples` تولید نمایید و آن را `noiseSamples` نامگذاری نمایید. سپس این بردار را با بردار `rxSamples` جمع نموده و حاصل را `rxSamplesPlusNoise` بنامید.

## آزمایش ۳-۴: پیاده‌سازی گیرنده‌ی مدولاسیون

در ادامه‌ی فایل آزمایش قبل می‌خواهیم گیرنده‌ی یک مدولاسیون خطی را پیاده‌سازی نماییم.

۵. مدولاسیون یا آشکارسازی پالس: نمونه‌های دریافت شده به همراه نویز (`rxSamplesPlusNoise`) را که اثر تأخیر کانال در آن جبران شده را درون بردار `rxSamples` قرار دهید. با استفاده از تابع `pulseDemodulation` با ورودی‌های نمونه‌های سیگنال دریافتی (`rxSamples`)، نام مدولاسیون (`modulation`)، مرتبه‌ی مدولاسیون (`M`)، نرخ نمونه‌برداری (`fs`)، تعداد نمونه‌های هر سمبل (`nSymbolSamples`)، نام تابع شکل‌دهنده‌ی پالس (`pulseName`) و روش تشخیص قدرت سیگنال دریافتی در راستای سیگنال‌های پایه (`mode`)، سمبل‌های دریافتی (`rxSymbols`) و اندیس سمبل‌های آشکارشده (`detectedSymbolsIndex`) را به دست آورید.

۶. تبدیل از کدگذاری گری به کدگذاری باینری: به کمک بردار `detectedSymbolsIndex` و ماتریس `grayMatrix` اندیس سمبل‌های آشکارشده‌ی با کدگذاری باینری را به دست آورید و درون متغیر `detectedBits` قرار دهید. برای این منظور متناظر با محتوای هر سطر `detectedSymbolsIndex` سطری متناظر با آن محتوا از `grayMatrix` انتخاب می‌شود. این عمل را می‌بایست برای حالتی که از کدگذاری باینری نیز استفاده می‌شود و `isGray = 0` است نیز مدیریت شود.

۷. محاسبه‌ی خطای سمبل: با مقایسه‌ی تعداد اختلاف‌های بردارهای `detectedSymbolsIndex` و `symbolIndex` و محاسبه نسبت اختلاف این دو بردار به تعداد کل سمبل‌ها، خطای سمبل را به دست آورده و آن را در متغیر `ser` قرار دهید.

۸. تبدیل سمبل به بیت و محاسبه‌ی خطای بیت: با مقایسه‌ی تعداد اختلاف ماتریس‌های `detectedBits` و `txBit` و محاسبه نسبت تعداد اختلاف این دو ماتریس به تعداد کل بیت‌ها، خطای بیت را به دست آورده و آن را در متغیر `ber` قرار دهید.

## آزمایش ۳-۵: خواسته‌های کلی

۱. رسم نمودار نرخ خطای بیت: با تغییر نسبت سیگنال به نویز بین 0 تا 10dB نرخ خطای بیت را برای مدولاسیون‌های 16QAM، 8PSK و 4PAM را به دست آورده و در یک نمودار رسم نمایید و آن را با خروجی تابع `berawgn` نرم‌افزار MATLAB مقایسه نمایید. این کار را برای شکل موج مثلثی و `root raised cosine` انجام دهید. هم‌چنین اثر اعمال کدگذاری گری و کدگذاری باینری را نیز بر روی نمودار نرخ خطای بیت نشان دهید.

۲. اثر تأخیر در کانال:

در این جا فرض نمایید مدولاسیون 4PSK است و نسبت سیگنال به نویز ( $E_b/N_0$ ) برابر با 10dB است.

گام ۱. اثر تأخیر در کانال بر روی نرخ خطای بیت و منظومه‌ی سیگنالی: تأخیر کانال را برابر با گرد شده‌ی 0.1، 0.5 و 0.8 برابر طول یک سمبل قرار دهید (یک عدد صحیح معادل تعداد نمونه‌های زمانی تأخیر یافته). نرخ خطای بیت را به دست آورد و با حالت بدون تأخیر مقایسه نمایید. منظومه سیگنالی حالت‌های فوق را نیز با یکدیگر مقایسه کرده و علت مشاهدات خود را توضیح دهید. (`rxReceiveMode = 0` و `isHardware = 0`)

گام ۲. اضافه کردن هدر: برای یافتن نقطه‌ی شروع درست داده‌ها، تعدادی داده‌ی مشخص به عنوان هدر در ابتدای رشته بیت‌های ارسالی قرار دهید. نمونه‌های زمانی مربوط به هدر را از بردار `txSamples` جدا کرده و در متغیر `headerSamples` ذخیره نمایید. بیت‌های هدر می‌بایست به صورت زیر باشد. این رشته بیت خواص خودهمبستگی خوبی دارد.

گام ۳. تخمین زمان شروع بسته و جبران اثر تأخیر کانال: با استفاده از همبستگی بردار `rxSamplesPlusNoise` و نمونه‌های زمانی هدر `headerSamples`، نقطه‌ی شروع بسته‌ی داده را تخمین بزنید. با مقایسه‌ی نرخ خطای



بیت حالت بدون تأخیر و حالتی که نقطه‌ی شروع داده‌ها تخمین زده می‌شود، از عملکرد برنامه‌ی خود اطمینان حاصل نمایید. (`rxReceiveMode = 1` و `isHardware = 0`)

### ۳. اعمال فاز در کانال و تخمین فاز:

در این جا فرض نمایید مدولاسیون 4PSK است و نسبت سیگنال به نویز ( $E_b/N_0$ ) برابر با 10dB است.

گام ۱. اثر فاز اضافی کانال بر نرخ خطای بیت: اختلاف فاز کانال را برابر ۳۰ درجه قرار دهید. ابتدا برای مدولاسیون 4PSK نرخ خطای بیت را به دست آورده و با حالت بدون اختلاف فاز مقایسه نمایید. منظومه سیگنالی هر دو حالت را نیز با یکدیگر مقایسه نمایید. علت مشاهدات خود را توضیح دهید.

گام ۲. تخمین فاز اضافی کانال: اگر به ابتدای بسته‌ی ارسالی هدر بخش قبل اضافه شود، با استفاده از همبستگی بردار `rxSamplesPlusNoise` و نمونه‌های زمانی هدر `headerSamples`، فازی را که کانال به سیگنال اضافه کرده است را به دست آورید. (راهنمایی: به فاز در قله‌ی خروجی همبستگی دقت نمایید. قله از روی مقدار مطلق همبستگی به دست می‌آید ولی مقدار خود همبستگی در این اندیس به دست آمده مورد استفاده قرار می‌گیرد. به عملکرد تابع `max` بر روی داده‌های مختلط توجه نمایید.)

گام ۳. جبران‌سازی فاز کانال: تلاش کنید با اعمال ضریب مناسب اثر فاز اضافی کانال را جبران نمایید. (`equalizerMode = 2`)

## آزمایش ۳-۶: پیاده‌سازی سخت‌افزاری

در این بخش `isHardware = 1` و `rxReceiveMode = 1` می‌باشد.

۱. ارسال نمونه‌ها با استفاده از ADALM-PLUTO: بردار `txSamples` مربوط به مدولاسیون 4PSK (هدر اضافه شود) را با استفاده از دستور `transmitRepeat` به مدت زمان `dataTransferDuration` ثانیه به صورت پی‌درپی در فضا ارسال نمایید. این دستور داده‌ها را از طریق USB به رادیو نرم‌افزار ارسال می‌نماید و در حافظه‌ی سخت‌افزار ذخیره می‌نماید. فرستنده داده‌ها را از حافظه‌ی رادیو نرم‌افزار قرائت کرده و مدام ارسال می‌نماید. برای این منظور ابتدا می‌بایست پس از پیکربندی رادیو نرم‌افزار یک شیء فرستنده ایجاد نموده و فرکانس مرکزی آن را بر روی 2400MHz تنظیم کرده و بهره‌ی آن را برابر با 0dB تنظیم نمایید.

۲. دریافت سیگنال با استفاده از ADALM-PLUTO: ابتدا شیء مربوط به گیرنده را تعریف نمایید. فرکانس مرکزی آن را بر روی 2400MHz تنظیم کرده و بهره‌ی آن را به صورت Manual و برابر 20dB تنظیم نمایید. به علت اطمینان از دریافت یک بسته‌ی کامل، تعداد نمونه‌های زمانی دریافتی را برابر با دو برابر تعداد نمونه‌های زمانی بسته در نظر بگیرید. به منظور دریافت داده، شیء گیرنده را فراخوانی نمایید و حاصل را درون متغیر `rxSamples` قرار دهید.

۳. اثر شکل پالس بر روی پهنای باند سیگنال ارسالی: طیف فرکانسی سیگنال `rxSamples` دریافتی از ADALM-PLUTO را برای مدولاسیون 4PSK و به ازای شکل پالس‌های مثلثی و root raised cosine رسم نماید. در مورد پهنای باند و سطح باندهای جانبی آن توضیحاتی ارائه دهید. برای این منظور از برنامه‌های قبلی استفاده نمایید.

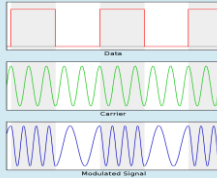
۴. دمدولاسیون و مشاهده‌ی منظومه‌ی سیگنالی: مشابه قبل عمل دمدولاسیون را انجام داده و منظومه‌ی سیگنالی مربوط به `rxSymbols` را رسم نمایید و مشاهدات خود را یادداشت نمایید. منظومه‌ی سیگنالی باید بتواند به صورت به‌لحظه به‌روز شود.

۵. جبران‌سازی دامنه، فاز و تأخیر و مشاهده‌ی منظومه‌ی سیگنالی: با یافتن ابتدای بسته‌ها و جبران فاز، دامنه و تأخیر، منظومه‌ی سیگنالی مربوط به `rxSymbols` را رسم نمایید. احتمال خطای بیت را نیز در این حالت محاسبه نمایید.



- [1] J. G. Proakis and M. Salehi, *Digital Communications*. Boston: McGraw-Hill, 2008.
- [2] M. Rice, *Digital Communications: A Discrete-Time Approach*. Upper Saddle River, NJ: Pearson Education, 2009.
- [3] R. W. Stewart, K. Barlee, L. Crockett, and D. Atkinson, *Software Defined Radio using MATLAB & Simulink and the RTL-SDR*. 2015.





# آزمایش چهارم

## مدولاسیون FSK همدوس

### اهداف آزمایش

- پس از آشنایی با مدولاسیون‌های خطی و اثر یک کانال AWGN بر روی آن، در این آزمایش با مدولاسیون FSK همدوس آشنا می‌شویم و رفتار آن را دوباره در کانال AWGN مورد بررسی قرار می‌دهیم.
- در انتهای این آزمایش دانشجو می‌بایست:
- با مفهوم مدولاسیون‌های متعامد آشنا شود.
  - منظومه‌ی سیگنالی مدولاسیون FSK را بشناسد.
  - با شکل موج مدولاسیون FSK آشنا باشد.
  - بر اثر فاصله‌ی فرکانسی بین پایه‌های مدولاسیون FSK مسلط شود.
  - بتواند یک سیستم کامل مدولاسیون FSK را پیاده‌سازی کند.
  - احتمال خطای سمبل و بیت را در این مدولاسیون به دست آورد.
  - یه سامانه‌ی کامل مدولاسیون FSK همدوس را به کمک رادیو نرم‌افزار ADALM-PLUTO پیاده‌سازی نماید.

### ابزارهای مورد نیاز

- رایانه
- نرم‌افزار MATLAB R2020b
- رادیو نرم‌افزار ADALM-PLUTO
- آنتن



## مدولاسیون‌های متعامد

در مدولاسیون‌های متعامد، بُعد فضا برابر با تعداد نقاط منظومه‌ی سیگنالی می‌باشد. به عبارتی، در صورتی که  $N$  نشان‌دهنده‌ی بُعد فضا باشد،  $N = M$  خواهد بود که  $S_1(t), S_2(t), \dots, S_M(t)$  سیگنال‌های ارسالی می‌باشند. برای این مدولاسیون‌ها داریم:

$$\langle S_i(t), S_j(t) \rangle = \begin{cases} 0 & i \neq j \\ E_s & i = j \end{cases}$$

بدیهی است که سیگنال‌های پایه برای این مدولاسیون‌ها به صورت زیر خواهند بود:

$$\phi_i(t) = \frac{S_i(t)}{\|S_i(t)\|} = \frac{S_i(t)}{\sqrt{E_s}}$$

حالت خاصی از مدولاسیون‌های متعامد، مدولاسیون FSK<sup>6</sup>، می‌باشد که سیگنال‌های ارسالی به صورت زیر می‌باشند:

$$S_m(t) = \sqrt{\frac{2E_s}{T}} \cos(2\pi(f_0 + (m-1)\Delta f)t) \quad 0 < t \leq T, \quad m = 1, 2, \dots, M$$

همان‌گونه که مشاهده می‌شود، شکل‌دهنده‌ی پالس در اینجا یک مستطیل به طول  $T$  خواهد بود. معادل باندپایه برای این سیگنال‌ها به صورت زیر خواهد بود.

$$S_{\ell,m}(t) = \sqrt{\frac{2E_s}{T}} e^{j2\pi(m-1)\Delta f t}$$

حال می‌خواهیم  $\Delta f$  را به گونه‌ای تنظیم کنیم که سیگنال‌های ارسالی بر یکدیگر عمود باشند.

$$\begin{aligned} \langle S_{\ell,m}(t), S_{\ell,n}(t) \rangle &= \int_{-\infty}^{\infty} S_{\ell,m}(t) S_{\ell,n}^*(t) dt = \int_0^T \sqrt{\frac{2E_s}{T}} e^{j2\pi(m-1)\Delta f t} \sqrt{\frac{2E_s}{T}} e^{-j2\pi(n-1)\Delta f t} dt = \frac{2E_s}{T} \int_0^T e^{j2\pi(m-n)\Delta f t} dt \\ &= \frac{2E_s}{T} e^{j\pi(m-n)\Delta f T} \times \frac{\sin(\pi(m-n)\Delta f T)}{\pi(m-n)\Delta f} \end{aligned}$$

از طرفی داریم:

$$\langle S_m(t), S_n(t) \rangle = \frac{1}{2} \text{Re}\{\langle S_{\ell,m}(t), S_{\ell,n}(t) \rangle\}$$

بنابراین:

$$\langle S_m(t), S_n(t) \rangle = \frac{1}{2} \text{Re}\{\langle S_{\ell,m}(t), S_{\ell,n}(t) \rangle\} = E_s \frac{(\cos(\pi(m-n)\Delta f T) \times \sin(\pi(m-n)\Delta f T))}{\pi(m-n)\Delta f T} = E_s \text{sinc}(2(m-n)\Delta f T)$$

به منظور برقراری شرط تعامد لازم است که:

$$\text{sinc}(2(m-n)\Delta f T) = 0$$

بنابراین برای اینکه تعامد در باند میانی برقرار شود لازم است که شرط  $\Delta f = \frac{k}{2(m-n)T}$ ;  $k \in \mathbb{Z}$  برقرار باشد. حداقل مقدار  $\Delta f$  که

تعامد را برقرار می‌سازد برابر مقدار زیر است.

$$\Delta f_{\min} = \frac{1}{2T}$$

از آنجا که پهنای باند مصرفی با  $\Delta f$  متناسب است، معمولاً در عمل سعی می‌شود که از  $\Delta f_{\min}$  استفاده شود. لازم به ذکر است که

پایه‌های فضای سیگنال در باند میانی به صورت زیر می‌باشند:

$$\Phi_m(t) = \sqrt{\frac{2}{T}} \times \cos(2\pi(f_0 + (m-1)\Delta f)t), \quad 0 < t \leq T$$

<sup>6</sup> Frequency Shift Keying



## شرح آزمایش

### آزمایش ۱-۴: مقداردهی‌های اولیه

در این آزمایش بنا داریم تعدادی بسته‌ی داده را برای ارسال از طریق مدولاسیون FSK آماده نماییم. از آن‌جا که در این آزمایش بناسست این فرستنده با استفاده از رادیو نرم‌افزار ADALM-PLUTO پیاده‌سازی شود، می‌بایست با استفاده از پارامترها و ملاحظات عملی، این کار انجام شود.

پارامترهایی که در این پیاده‌سازی اهمیت دارد شامل نرخ نمونه‌برداری ( $f_s$ )، تعداد نمونه‌های هر سمبل ( $nSymbolSamples$ )، تعداد سمبل‌های ارسالی در بسته‌ی داده ( $nPacketSymbols$ ) و مدت زمان ارسال داده ( $dataTransferDuration$ ) می‌باشد. در این آزمایش مقدار پارامترهای مورد نیاز به صورت جدول ۳ می‌باشد.

جدول ۳ پارامترهای آزمایش ۶

| پارامتر | $f_s$ | $nSymbolSamples$ | $nPacketSymbols$                     | $dataTransferDuration$ |
|---------|-------|------------------|--------------------------------------|------------------------|
| مقدار   | 10MHz | 64               | شبیه‌سازی: ۱۰۰۰۰۰<br>سخت‌افزار: ۱۰۰۰ | ۱۰۰ ثانیه              |

دقت نظر داشته باشید که در این آزمایش نیز تنها می‌خواهیم برنامه‌هایی که در آزمایش قبل نوشته شده است را تکمیل نماییم.

۱. تولید M-File مربوط به مقدار دهی اولیه: قالب کلی پارامترهای مورد نیاز در این آزمایش، مانند آزمایش قبل است و فایل `dclInitialize.m` می‌بایست مدولاسیون FSK را نیز بشناسد. این فایل می‌بایست از برنامه‌ی اصلی فراخوانی نماییم.

### آزمایش ۲-۴: پیاده‌سازی فرستنده مدولاسیون FSK

۱. تولید بیت: با استفاده از تابع `bitGenerator` تعدادی بیت ۰ و ۱ را متناسب با طول بسته‌ی داده و با در نظر گرفتن پارامترهای مدولاسیون FSK 4 تولید نموده و آن را درون ماتریس `txBit` ذخیره نماییم. ماتریس `txBit` یک ماتریس با اندازه‌ی  $nPacketSymbols \times k$  می‌باشد.

۲. نگاشت بیت به شماره‌ی سمبل: بردار جدیدی متناظر با ماتریس `txBit` و با نام `symbolIndex` تولید نماییم و متناظر با هر سطر ماتریس `txBit` یک عدد صحیح معادل با آن بیت‌ها در `symbolIndex` قرار دهیم. دقت نماییم در مدولاسیون FSK نیازی به کدگذاری گری نیست. بنابراین در برنامه از پرچم `isGray` برای اعمال یا عدم اعمال کدگذاری گری استفاده نماییم.

۳. تولید نمونه‌های زمانی سمبل ارسالی: با استفاده از تابع `pulseModulation` ابتدا نمونه‌های زمانی متناظر با هر شماره‌ی سمبل را تولید و در نهایت نمونه‌های زمانی یک بسته‌ی داده را آماده‌ی ارسال کرده و حاصل را در بردار `txSamples` ذخیره نماییم. دقت نماییم که انرژی هر سمبل برابر با  $E_s = 1$  باشد تا انرژی متوسط سمبل‌ها نیز برابر با  $E_{s,avg} = 1$  شود.

توضیحات تکمیلی: در این آزمایش نمونه‌های سیگنال در باندپایه ایجاد می‌شود. چون سیگنال در باند میانی تولید نمی‌شود نیازی به دانستن  $f_0$  نیست. از این رو می‌بایست از فرمول معادل باندپایه‌ی سیگنال مدولاسیون FSK استفاده نمود. فرمول‌هایی که در بخش توضیحات این آزمایش نوشته شده مربوط به حالت سیگنال پیوسته است. در حالت گسسته ابتدا یک بردار زمان به صورت  $t = (0:nSymbolSamples-1) * t_s$  ایجاد نمود که در آن  $t_s = 1/f_s$  زمان نمونه‌برداری می‌باشد. سپس با استفاده از این بردار که در واقع نقاط نمونه‌برداری از معادل پایین‌گذر است، شکل گسسته‌ی هر کدام از سمبل‌ها به دست می‌آید. ضریب مناسبی به جای  $\sqrt{\frac{2E_s}{T}}$  می‌بایست گذاشته شود تا انرژی هر سمبل واحد شود. حال نمونه‌های سمبل‌های مختلف را پشت سر هم قرار می‌دهیم. در واقع اگر تعداد سمبل‌ها برابر ۱۰ باشد و  $nSymbolSamples$  برابر

با باشد، بردار **txSamples** یک بردار با ۶۴۰ سطر و یک ستون خواهد بود. با استفاده از یک حلقه `for` می‌توان پس از تولید هر یک از سمبل‌ها آن را در موقعیت مناسب در بردار **txSamples** قرار داد.

### آزمایش ۳-۴: مدل سازی کانال

۱. افزودن تأخیر در کانال: برای شبیه‌سازی تأخیر در کانال به اندازه‌ی پارامتر **channelDelayInSample** به ابتدای بردار **txSamples** صفر اضافه نمایید. حاصل را درون بردار **txSamplesDelayed** قرار دهید. دقت نمایید که طول بردار افزایش می‌باید و می‌بایست در ادامه‌ی برنامه اثر این افزایش لحاظ شود. در این جا فعلاً مقدار تأخیر برابر با صفر قرار دهید.
۲. اعمال اختلاف فاز در کانال: برای اعمال اختلاف فاز در کانال می‌بایست بردار **txSamplesDelayed** را در  $\exp(1i \cdot \text{channelPhaseOffset})$  ضرب شود. حاصل را در بردار **rxSamples** قرار دهید. در این جا نیز آفست فاز را برابر با صفر قرار دهید.
۳. شبیه‌سازی کانال با نویز سفید گاوسی
  ۱. تعیین واریانس نویز بر اساس نسبت سیگنال به نویز ( $E_b/N_0$ ): با توجه به تولید هر سمبل با انرژی واحد، مقدار انرژی متوسط بیت را به دست آورده و برابر متغیر **bitAverageEnergy** قرار دهید. مقدار  $E_b/N_0$  را برابر با 10dB در نظر بگیرید (به عبارتی پارامترهای **snrMin** و **snrMax** برابر 10 تنظیم می‌شوند). سپس واریانس نویز را بر اساس نسبت  $E_b/N_0$  به دست آورید و آن را درون متغیر **noiseVariance** قرار دهید.
  ۲. افزودن نویز به سیگنال: با استفاده از تابع **randn** یک بردار نویز مختلط با واریانس **noiseVariance** و ابعاد برابر با **txSamples** تولید نمایید و آن را **noiseSamples** نامگذاری نمایید. سپس این بردار را با بردار **rxSamples** جمع نموده و حاصل را **rxSamplesPlusNoise** بنامید.

### آزمایش ۴-۴: پیاده‌سازی گیرنده‌ی مدولاسیون FSK

در این جا می‌خواهیم گیرنده‌ی مدولاسیون FSK خطی را پیاده‌سازی نماییم.

۱. دمدولاسیون یا آشکارسازی پالس: نمونه‌های ارسال شده به همراه نویز (**txSamplesPlusNoise**) را به عنوان نمونه‌های دریافتی درون بردار **rxSamples** قرار دهید. با استفاده از تابع **pulseDemodulation** با ورودی‌های نمونه‌های سیگنال دریافتی (**rxSamples**)، نام مدولاسیون (**modulation**)، مرتبه‌ی مدولاسیون (**M**)، نرخ نمونه‌برداری (**fs**)، تعداد نمونه‌های هر سمبل (**nSymbolSamples**)، نام تابع شکل‌دهنده‌ی پالس (**pulseName**) و روش تشخیص قدرت سیگنال دریافتی در راستای سیگنال‌های پایه (**mode**)، خروجی سمبل‌های خروجی (**rxSymbols**) و اندیس سمبل‌های آشکار شده (**detectedSymbolsIndex**) را به دست آورید. (در این جا به پارامترهای ورودی **mode**، **pulseName**، **detectedSymbolsIndex** نیاز نیست).
۲. محاسبه‌ی خطای سمبل: با مقایسه‌ی تعداد اختلاف‌های بردارهای **detectedSymbolsIndex** و **symbolIndex** و محاسبه نسبت اختلاف این دو بردار به تعداد کل سمبل‌ها، خطای سمبل را به دست آورده و آن را در متغیر **ser** قرار دهید.
۳. تبدیل سمبل به بیت و محاسبه‌ی خطای بیت: بیت‌های متناظر با سطرهای **detectedSymbolsIndex** را به دست آورید و آن را درون متغیر **detectedBits** قرار دهید. با مقایسه‌ی تعداد اختلاف ماتریس‌های **detectedBits** و **txBit** و محاسبه نسبت تعداد اختلاف این دو ماتریس به تعداد کل بیت‌ها، خطای بیت را به دست آورده و آن را در متغیر **ber** قرار دهید.

## آزمایش ۴-۵: خواسته‌های کلی

۱. رسم نمودار نرخ خطای بیت: با تغییر نسبت سیگنال به نویز بین 0 تا 10dB نرخ خطای بیت را برای مدولاسیون‌های 2FSK، 4FSK و 8FSK را به دست آورده و در نمودارهایی جداگانه رسم نمایید و آن را با خروجی تابع **berawgn** مقایسه نمایید.

۲. اثر تأخیر در کانال:

در این جا فرض نمایید مدولاسیون 2FSK است و نسبت سیگنال به نویز ( $E_b/N_0$ ) را از 0dB تا 10dB تغییر دهید.

گام ۱. اثر تأخیر در کانال بر روی نرخ خطای بیت و منظومه‌ی سیگنالی: تأخیر کانال را برابر با گرد شده‌ی 0.1، 0.5 و 0.8 برابر طول یک سمبل قرار دهید (یک عدد صحیح معادل تعداد نمونه‌های زمانی تأخیر یافته). نمودار نرخ خطای بیت را به دست آورد و با حالت بدون تأخیر مقایسه نمایید. منظومه سیگنالی هر دو حالت فوق را نیز با یکدیگر مقایسه نمایید. علت مشاهدات خود را توضیح دهید.

گام ۲. اضافه کردن هدر: برای یافتن نقطه‌ی شروع درست داده‌ها، تعدادی داده‌ی مشخص به عنوان هدر در ابتدای رشته بیت‌های ارسالی قرار دهید. نمونه‌های زمانی مربوط به هدر را از بردار **txSamples** جدا کرده و در متغیر **headerSamples** ذخیره نمایید. بیت‌های هدر می‌بایست به صورت زیر باشد. این رشته بیت خواص خودهمبستگی خوبی دارد.

گام ۳. تخمین زمان شروع بسته و جبران اثر تأخیر کانال: با استفاده از همبستگی بردار **txSamples\_noisetxSamplesPlusNoise** و نمونه‌های زمانی هدر **headerSamples**، نقطه‌ی شروع بسته‌ی داده را تخمین بزنید. با مقایسه‌ی نرخ خطای بیت حالت بدون تأخیر و حالتی که نقطه‌ی شروع داده‌ها تخمین زده می‌شود، از عملکرد برنامه‌ی خود اطمینان حاصل نمایید.

۳. اعمال فاز در کانال و تخمین فاز:

در این جا فرض نمایید مدولاسیون 2FSK است و نسبت سیگنال به نویز ( $E_b/N_0$ ) را از 0dB تا 10dB تغییر دهید.

گام ۱. اثر فاز اضافی کانال بر نرخ خطای بیت: اختلاف فاز کانال را برابر ۳۰ درجه قرار دهید. ابتدا نمودار نرخ خطای بیت را به دست آورده و با حالت بدون اختلاف فاز مقایسه نمایید. منظومه سیگنالی هر دو حالت فوق را نیز با یکدیگر مقایسه نمایید. علت مشاهدات خود را توضیح دهید.

گام ۲. تخمین فاز اضافی کانال: اگر به ابتدای بسته‌ی ارسالی هدر بخش قبل اضافه شود، با استفاده از همبستگی بردار **rxSamplesPlusNoise** و نمونه‌های زمانی هدر **headerSamples**، فازی را که کانال به سیگنال اضافه کرده است را به دست آورید. (راهنمایی: به فاز در قله‌ی خروجی همبستگی دقت نمایید.)

گام ۳. جبران‌سازی فاز کانال: تلاش کنید با اعمال ضریب مناسب اثر فاز اضافی کانال را جبران نمایید. (**equalizerMode = 2**)

## آزمایش ۴-۷: پیاده‌سازی سخت‌افزاری گیرنده و فرستنده‌ی FSK

در این بخش **isHardware = 1** و **rxReceiveMode = 1** می‌باشد.

۱. ارسال نمونه‌ها با استفاده از ADALM-PLUTO: بردار **txSamples** مربوط به مدولاسیون 2FSK (هدر اضافه شود) را با استفاده از دستور **transmitRepeat** به مدت زمان **dataTransferDuration** ثانیه به صورت پی‌درپی در فضا ارسال نمایید. این دستور داده‌ها را از طریق USB به رادیو نرم‌افزار ارسال می‌نماید و در حافظه‌ی سخت‌افزار ذخیره می‌نماید. فرستنده داده‌ها را از حافظه‌ی رادیو نرم‌افزار قرائت کرده و مدام ارسال می‌نماید. برای این منظور ابتدا می‌بایست پس از پیکربندی رادیو نرم‌افزار یک شیء فرستنده ایجاد نموده و فرکانس مرکزی آن را بر روی 2400MHz تنظیم کرده و بهره‌ی آن را برابر با 0dB تنظیم نمایید.

۲. دریافت سیگنال با استفاده از ADALM-PLUTO: ابتدا شیء مربوط به گیرنده را تعریف نمایید. فرکانس مرکزی آن را بر روی 2400MHz تنظیم کرده و بهره‌ی آن را به صورت Manual و برابر 20dB تنظیم نمایید. به علت اطمینان از دریافت یک بسته‌ی کامل، تعداد نمونه‌های زمانی دریافتی را برابر با دو برابر تعداد نمونه‌های زمانی بسته در نظر بگیرید. به منظور دریافت داده، شیء گیرنده را فراخوانی نمایید و حاصل را درون متغیر **rxSamples** قرار دهید.

۳. طیف سیگنال ارسالی: طیف فرکانسی سیگنال **rxSamples** دریافتی از ADALM-PLUTO را برای مدولاسیون 2FSK را رسم نماید. در مورد پهنای باند و سطح باندهای جانبی آن توضیحاتی ارائه دهید. برای این منظور از برنامه‌های قبلی استفاده نمایید.

۴. دمدولاسیون و مشاهده‌ی منظومه‌ی سیگنالی:

- گام ۱. مشابه قبل عمل دمدولاسیون را انجام داده و منظومه‌ی سیگنالی مربوط به **rxSymbols** را رسم نمایید. منظومه‌ی سیگنالی باید بتواند به صورت به لحظه به روز شود.
- گام ۲. علت تفاوت منظومه‌ی سیگنالی در هنگام کار با سخت افزار را با حالت شبیه سازی را با مشاهده‌ی تفاوت شکل موج سیگنال دریافتی در حالت کار با سخت افزار و شبیه سازی توضیح دهید.
- گام ۳. با تغییر ویژگی‌های زیر برای سخت افزار مجدد مشاهدات خود را بیان نمایید. (توضیح بیشتر با جستجوی عبارت DC Offset Tracking در راهنمای نرم افزار MATLAB)

```
rx.ShowAdvancedProperties = true;  
rx.EnablebitAverageEnergyasbitAverageEnergyandDCCorrection = false;
```

۵. جبران سازی دامنه، فاز و تأخیر و مشاهده‌ی منظومه‌ی سیگنالی: با یافتن ابتدای بسته‌ها و جبران فاز، دامنه و تأخیر، منظومه‌ی سیگنالی مربوط به **rxSymbols** را رسم نمایید. احتمال خطای بیت را نیز در این حالت محاسبه نمایید.

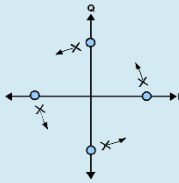
۶. اثر  $\Delta f$  بر روی پهنای باند ارسالی: طیف فرکانسی سیگنال **rxSamples** دریافتی از ADALM-PLUTO را برای مدولاسیون 4FSK رسم نماید. در مورد پهنای باند و سطح باندهای جانبی آن توضیحاتی ارائه دهید. با تغییر  $\Delta f$  از  $\frac{1}{2T}$  به  $\frac{2}{T}$  پهنای باند سیگنال دریافتی هر حالت را به دست آورده و با یکدیگر مقایسه نمایید.



- [1] J. G. Proakis and M. Salehi, *Digital Communications*. Boston: McGraw-Hill, 2008.
- [2] R. W. Stewart, K. Barlee, L. Crockett, and D. Atkinson, *Software Defined Radio using MATLAB & Simulink and the RTL-SDR*. 2015.







# آزمایش پنجم

## آشکارسازی ناهمدوس

### اهداف آزمایش

در طول آزمایش‌های قبل تمرکز بر روی آشکارسازی همدوس بوده است. در صورت وجود تأخیر کانال و تغییر فاز سیگنال دریافتی، ابتدا این فاز جبران و سپس آشکارسازی صورت می‌پذیرفت. در این آزمایش با مدولاسیون‌هایی آشنا می‌شویم که با وجود جبران نکردن فاز می‌توان عمل آشکارسازی را انجام داد که به این آشکارسازی، آشکارسازی ناهمدوس گفته می‌شود.

در انتهای این آزمایش دانشجو می‌بایست:

- با مفهوم آشکارسازی ناهمدوس آشنا شود. 
- آشکارسازی ناهمدوس مدولاسیون FSK را انجام دهد. 
- با نحوه‌ی تولید و آشکارسازی مدولاسیون DBPSK آشنا شود. 
- با شبیه‌سازی کانال با فاز تصادفی آشنا باشد. 
- این نوع آشکارسازی را با سخت‌افزار پیاده‌سازی نماید. 

### ابزارهای مورد نیاز

- رایانه 
- نرم‌افزار MATLAB R2020b 
- رادیو نرم‌افزار ADALM-PLUTO 
- کابل کوکسیال 
- آنتن 



## آشکارسازی ناهمدوس

سیگنال پس از عبور از کانال دچار تأخیر زمانی می‌شود. این تأخیر زمانی باعث ایجاد ابهام فاز در سیگنال دریافتی می‌گردد. به عنوان مثال، سیگنال مربوط به مدولاسیون BPSK به صورت زیر است.

$$S_m(t) = \pm g(t) \cos(2\pi f_0 t) \equiv A_m g(t) \cos(2\pi f_0 t), \quad A_m \in \{\pm 1\}$$

در این جا بُعد فضا برابر با ۱ است و پایه‌ی این فضا به صورت زیر می‌باشد.

$$\Phi(t) = \sqrt{\frac{2}{E_g}} g(t) \cos(2\pi f_0 t)$$

پس از اضافه شدن نویز، سیگنال دریافتی در گیرنده به صورت زیر خواهد بود.

$$r(t) = S_m(t) + n(t) = \pm g(t) \cos(2\pi f_0 t) + n(t)$$

بنابراین برای آشکارسازی کافی است که در هر بازه‌ی  $T_s$  ضرب داخلی  $\phi(t)$  و  $r(t)$  را محاسبه شده و با توجه به علامت آن تصمیم‌گیری انجام شود. این نحوه‌ی تصمیم‌گیری در ادامه آمده است.

$$\hat{A}_m = \text{sgn}(\langle r(t), \Phi(t) \rangle) = \text{sgn}\left(\int r(t)\Phi(t)dt\right)$$

در بسیاری از موارد، سیگنال ارسالی پس از عبور از کانال دچار تأخیر زمانی می‌شود و سیگنال دریافتی در گیرنده به صورت زیر خواهد بود.

$$r(t) = S_m(t - \tau) + n(t) = \pm g(t) \cos(2\pi f_0 t + \phi) + n(t)$$

در رابطه‌ی فوق  $\phi = -2\pi f_0 \tau$  می‌باشد.

معمولاً این تأخیر، در مقایسه با پهنای پالس ارسالی ناچیز است. اما تأثیر این تأخیر بر روی فاز کاملاً قابل ملاحظه است. در صورتی که از تأثیر این تأخیر زمانی بر روی  $g(t)$  صرف نظر کنیم، سیگنال پایه‌ی فضا به صورت زیر خواهد بود.

$$\Phi(t) = \sqrt{\frac{2}{E_g}} g(t) \cos(2\pi f_0 t + \phi)$$

بنابراین برای تشکیل بردار پایه می‌بایست مقدار  $\phi$  را داشته باشیم که نیاز به تخمین  $\tau$  دارد. فرض کنید که مقدار  $\tau = 10^{-5} \text{ s}$

باشد. فرآیند تخمین زدن همواره با خطا همراه است و این خطا را به عنوان مثال ۱٪ در نظر می‌گیریم. بنابراین مقدار  $\tau$  تخمین زده شده برابر است با

$$\hat{\tau} = 10^{-5} \pm 10^{-7} \text{ s}$$

فرکانس حامل ( $f_0$ ) در سامانه‌های مختلف، مقادیر متفاوتی دارد. به عنوان مثال در شبکه‌ی سلولی مقدار فرکانس حامل از مرتبه‌ی

$10^9 \text{ Hz} \equiv 1 \text{ GHz}$  می‌باشد. بدین ترتیب  $\phi$  تخمین زده شده در گیرنده به صورت زیر خواهد بود.

$$\hat{\phi} = -2\pi f_0 \hat{\tau} = -2\pi f_0 (10^{-5} \pm 10^{-7}) = \phi \pm 2\pi \times 10^2$$

همان گونه که ملاحظه می‌شود، با وجود خطای بسیار کوچک در تخمین، خطای قابل ملاحظه‌ای در  $\phi$  حاصل خواهد شد.

در چنین مواردی بهتر است به جای تخمین پارامتر مبهم، آن را به صورت یک پارامتر غیر یقینی وارد محاسبات کرده و آن را در تصمیم‌گیری وارد نمود. برای مثال می‌توان  $\phi$  را به عنوان یک متغیر تصادفی با توزیع یکنواخت بین  $[0, 2\pi)$  در نظر گرفت و تصمیم‌گیری بهینه را انجام داد. به چنین فرآیندی آشکارسازی ناهمدوس گفته می‌شود.

فرض کنید که مجموعه سیگنال‌های ارسالی به صورت  $S_m(t) = g_m(t) \cos(2\pi f_0 t)$  باشد.  $S_m(t, \phi)$  را به صورت زیر تعریف

می‌کنیم

$$S_m(t, \phi) \triangleq g_m(t) \cos(2\pi f_0 t + \phi)$$

معادل باند پایه سیگنال  $S_m(t)$  را با  $S_{\ell, m}(t)$  نشان می‌دهیم. بنابراین:

$$S_m(t) = \Re\{S_{\ell, m}(t)e^{j2\pi f_0 t}\} \leftrightarrow S_m(t, \phi) = \Re\{S_{\ell, m}(t)e^{j2\pi f_0 t + j\phi}\}$$

معادل باند پایه را برای این سیستم به صورت زیر در نظر می‌گیریم:

$$r_l(t) = S_{\ell, m}(t)e^{j\phi} + n_l(t)$$

با فرض اینکه  $\phi$  یک متغیر تصادفی با توزیع یکنواخت بین  $[0, 2\pi)$  باشد، خواهیم داشت.



$$\hat{m} = \arg \max \left( \pi_m e^{\frac{-E_{\ell,m}}{2N_0}} I_0 \left( \frac{|r_{\ell} \cdot \underline{S}_{\ell,m}|}{N_0} \right) \right)$$

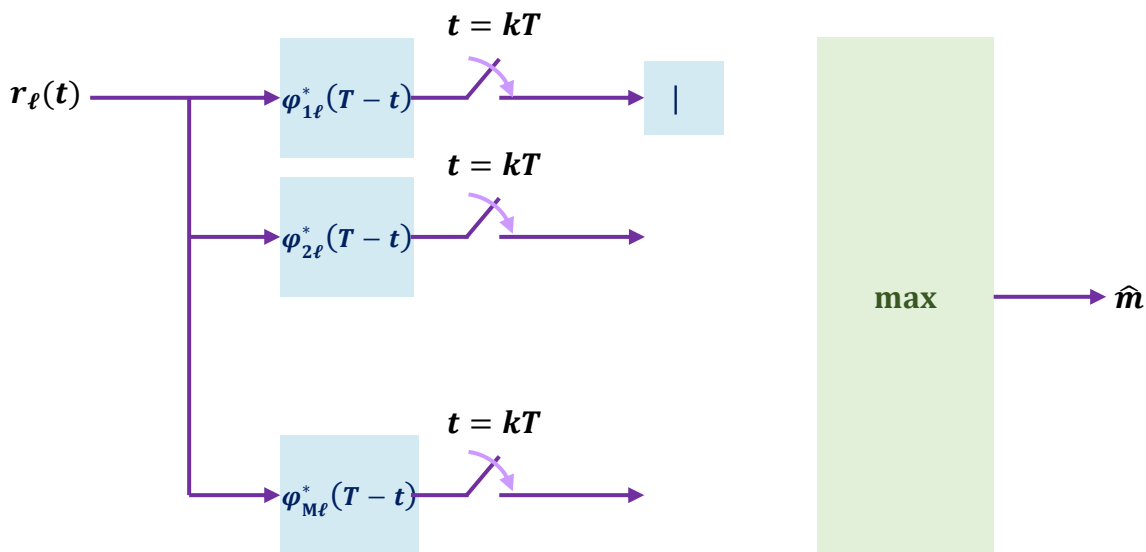
در رابطه‌ی فوق  $r_{\ell}$  معادل برداری سیگنال دریافتی در باند پایه،  $\underline{S}_{\ell,m}$  معادل برداری سیگنال ارسالی  $m$ ام در باند پایه و  $\pi_m$  احتمال ارسال سمبل  $m$ ام می‌باشد.  $I_0(x)$  نیز تابع بسل اصلاح شده مرتبه 0 است. این تابع نسبت به  $x$  متقارن است و به ازای مقادیر مثبت  $x$  صعودی نیز می‌باشد. عملگر  $(\cdot)$  معادل ضرب داخلی دو بردار است که به صورت زیر تعریف می‌شود.

$$\underline{X} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix}, \underline{Y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_N \end{bmatrix} \rightarrow \underline{X} \cdot \underline{Y} = \underline{Y}^H \underline{X}$$

بدین ترتیب اگر احتمال ارسال همه سمبل‌ها برابر و انرژی سیگنال ارسالی متناظر با هر کدام از آن‌ها یکسان باشد، عبارت فوق به صورت زیر ساده خواهد شد

$$\hat{m} = \arg \max (|r_{\ell} \cdot \underline{S}_{\ell,m}|)$$

گیرنده متناظر با این تصمیم‌گیری به صورت زیر خواهد بود.

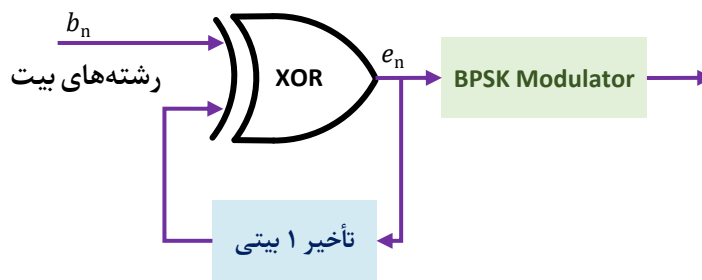


شکل 11 شمای گیرنده‌ی ناهمدوس

### مدولاسیون DPSK (Differential PSK)

سیگنال ارسالی در مدولاسیون PSK در حالت کلی به صورت  $g(t) \cos \left( 2\pi f_0 t + \frac{2\pi}{M} (m-1) \right)$  است. بنابراین تأخیر زمانی ایجاد شده به دلیل غیرایده‌آل بودن کانال، باعث می‌شود که سیگنال دریافتی به صورت  $g(t) \cos \left( 2\pi f_0 t + \frac{2\pi}{M} (m-1) + \phi \right)$  باشد. بنابراین نیاز است که آشکارسازی به صورت ناهمدوس صورت بگیرد. برای سادگی کار فرض می‌کنیم که مدولاسیون به ازای  $M = 2$  (BPSK) انجام پذیرفته باشد. احتمال خطای آشکارسازی ناهمدوس برای مدولاسیون BPSK برابر  $\frac{1}{2}$  است. این احتمال خطا نشان‌دهنده این واقعیت است که در صورت وقوع ابهام فاز در مدولاسیون BPSK، نمی‌توان در مورد سمبل ارسالی تصمیم‌گیری نمود و این مدولاسیون عملاً ناکارآمد خواهد شد.

برای مقابله با چنین اتفاقی، مدولاسیون DPSK (در حالت خاص DBPSK) پیشنهاد می‌شود. سمبل‌های ارسالی ما به صورت 0 و 1 ( $\pm 1$ ) خواهد بود. در این مدولاسیون، به ازای وقوع هر 1، فاز سیگنال ارسالی  $\pi$  رادیان و به ازای وقوع هر 0، فاز سیگنال ارسالی 0 رادیان تغییر خواهد کرد. بدین ترتیب در گیرنده برای آشکارسازی می‌بایست اختلاف فاز سیگنال فعلی را با سیگنال قبل به دست آوریم و تصمیم‌گیری انجام دهیم. این کار باعث می‌شود که ابهام  $\phi$  در فاز حذف شود و عملکرد مدولاسیون BPSK بهبود یابد. همان‌گونه که ملاحظه می‌شود، این مدولاسیون یک مدولاسیون حافظه‌دار است که برای تصمیم‌گیری برای سمبل فعلی، به سمبل قبلی نیاز خواهد بود. در عمل می‌توان برای پیاده‌سازی مدولاتور این مدولاسیون به صورت زیر عمل کرد.

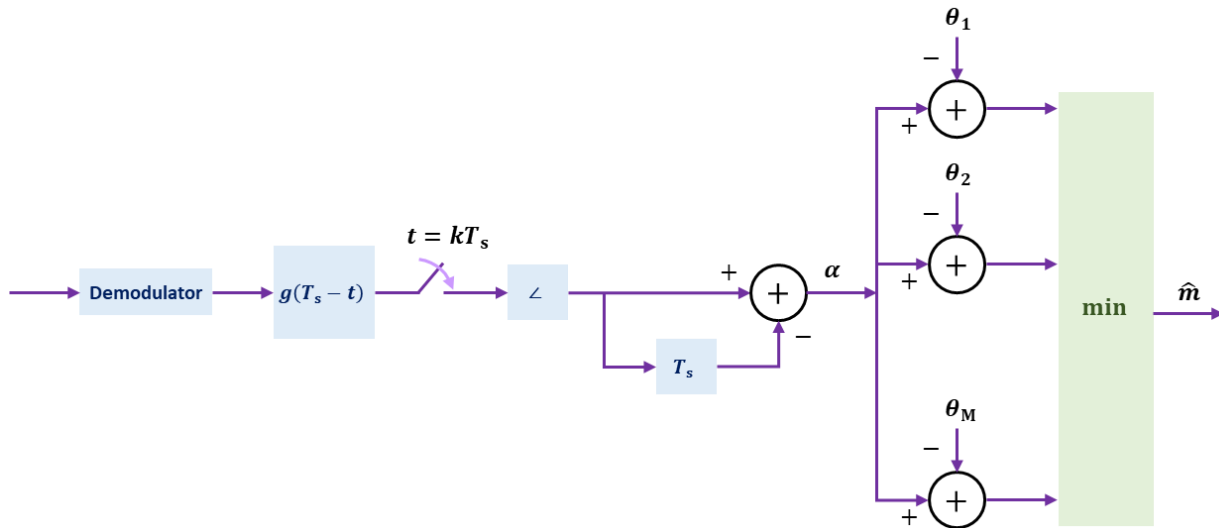


شکل 12 شمای مدولاسیون DPSK

بنابراین برای پیاده‌سازی مدولاتور DBPSK، ابتدا رشته بیت ورودی ( $b_n$ ) را به صورت زیر کدگذاری می‌کنیم.

$$e_k = e_{k-1} \oplus b_k$$

سپس با توجه به رشته بیت تولید شده، به ازای هر 0، فاز 0 و هر 1، فاز  $\pi$  را در نظر گرفت. دمدولاتور DPSK نیز به صورت زیر می‌باشد.



شکل 13 شمای گیرنده‌ی مدولاسیون DPSK

که برای حالت خاص  $m = 2$  می‌توان به صورت زیر تصمیم‌گیری نمود

$$\Re\{r_{\ell,k} r_{\ell,k-1}^*\} > 0 \rightarrow \hat{b}_k = 0$$

$$\Re\{r_{\ell,k} r_{\ell,k-1}^*\} < 0 \rightarrow \hat{b}_k = 1$$

$r_{\ell,k}$  معادل برداری سیگنال پایه دریافتی در  $k$  امین بازه‌ی سمبل ( $T_s$ ) است.

## شرح آزمایش

### آزمایش ۱-۵: مقداردهی‌های اولیه

در این آزمایش بنا داریم تعدادی بسته‌ی داده را برای ارسال از طریق مدولاسیون FSK ناهمدوس و DBPSK آماده نماییم. از آن جا که در این آزمایش بناست این فرستنده با استفاده از رادیو نرم‌افزار ADALM-PLUTO پیاده‌سازی شود، می‌بایست با استفاده از پارامترها و ملاحظات عملی، این کار انجام شود.

پارامترهایی که در این پیاده‌سازی اهمیت دارد شامل نرخ نمونه‌برداری ( $f_s$ )، تعداد نمونه‌های هر سمبل ( $nSymbolSamples$ )، تعداد سمبل‌های ارسالی در بسته‌ی داده ( $nPacketSymbols$ ) و مدت زمان ارسال داده ( $dataTransferDuration$ ) می‌باشد. در این آزمایش مقدار پارامترهای مورد نیاز به صورت جدول 3 می‌باشد.

جدول 4 پارامترهای آزمایش ۷

| پارامتر | $f_s$ | $nSymbolSamples$    | $nPacketSymbols$                     | $dataTransferDuration$ |
|---------|-------|---------------------|--------------------------------------|------------------------|
| مقدار   | 10MHz | FSK: 64<br>DBPSK: 8 | شبیه‌سازی: ۱۰۰۰۰۰<br>سخت‌افزار: ۱۰۰۰ | ۱۰۰ ثانیه              |

دقت نظر داشته باشید که در این آزمایش نیز تنها می‌خواهیم برنامه‌هایی که در آزمایش قبل نوشته شده است را تکمیل نماییم.

۱. تولید M-File مربوط به مقدار دهی اولیه: قالب کلی پارامترهای مورد نیاز در این آزمایش، مانند آزمایش قبل است و فایل `dclInitialize.m` می‌بایست پارامتری به نام `detectionMode` داشته باشد که نوع آشکارسازی را که می‌تواند همدوس (`coherent`) یا ناهمدوس (`noncoherent`) باشد را مشخص می‌نماید. برنامه با توجه به این گزینه نوع گیرنده و نحوه‌ی ارسال فرستنده مدیریت می‌شود. هم‌چنین پارامتر `channelPhaseMode` برای نوع اعمال ابهام فاز در نظر گرفته شود. این فایل می‌بایست از برنامه‌ی اصلی فراخوانی نمایید.

### آزمایش ۲-۵: پیاده‌سازی مدولاسیون FSK ناهمدوس

۱. شبیه‌سازی کانال با فاز تصادفی: برای شبیه‌سازی کانال با فاز تصادفی می‌بایست بردار `txSamplesDelayed` را در `exp(1i*channelPhase)` ضرب شود. حاصل را در بردار `txSamples` قرار دهید. `channelPhase` را به صورت یک بردار تصادفی بین ۰ و  $2\pi$  تولید کنید. طول این بردار برابر با طول بردار `txSamples` می‌باشد. دقت کنید که فاز اضافه شده در طول هر سمبل ثابت باشد. در این حالت `channelPhaseMode` برابر با ۰ است. (راهنمایی: از دستور `kron` می‌توان برای این کار استفاده نمود).

۲. شبیه‌سازی مدولاسیون FSK ناهمدوس: برای پیاده‌سازی این مدولاسیون، همانند FSK همدوس که در آزمایش قبل پیاده شد عمل نمایید. نمودار احتمال خطای بیت را برای مدولاسیون 4FSK و 2FSK به دست آورده و نتیجه‌ی را با دستور `berawgn` صحت‌سنجی نمایید. دقت نمایید در حالت آشکارسازی ناهمدوس باید حداقل فاصله‌ی فرکانسی  $1/T_s$  باشد. آشکارساز این مدولاسیون نیز به صورت  $\arg \max |r_{\ell} \cdot S_{\ell,m}|$  می‌باشد.

۳. پیاده‌سازی سخت‌افزاری مدولاسیون FSK ناهمدوس: با فرض این که توان فرستنده‌ی ADALM-PLUTO برابر با 0dBm و بهره‌ی گیرنده برابر با 20dB باشد و از آنتن به منظور ارتباط فرستنده و گیرنده استفاده می‌شود، خطای آشکارسازی با استفاده از آشکارسازی ناهمدوس را برای مدولاسیون 2FSK محاسبه نمایید. هم‌چنین در این حالت منظومه‌ی سیگنالی این مدولاسیون را نیز رسم نمایید. در این جا از هدر تنها برای به دست آوردن ابتدای بسته‌ی ارسالی استفاده می‌شود و هیچ گونه جبران‌سازی فازی صورت نمی‌پذیرد.

## آزمایش ۳-۵: پیاده‌سازی مدولاسیون DBPSK

- تولید بیت: با استفاده از تابع `bitGenerator` تعدادی بیت ۰ و ۱ را متناسب با طول بسته‌ی داده و با در نظر گرفتن پارامترهای مدولاسیون DBPSK تولید نموده و آن را درون ماتریس `txBit` ذخیره نمایید. ماتریس `txBit` یک ماتریس با اندازه‌ی  $nPacketSymbols \times 1$  می‌باشد.
- کد کردن بیت‌ها: با استفاده از شکل 12 رشته بیت‌های  $e_n$  را تولید نمایید و آن را درون بردار `encodedTxBits` قرار دهید.
- نگاشت بیت به سمبل:
  - کدگذاری گری بیت‌های تولیدی: با وجود این که در این جا نیازی به کدگذاری گری نیست ولی به منظور کلی بودن برنامه این مرحله نیز انجام می‌شود. با استفاده از تابع `grayMatrixGenerator`، ماتریس مربوط به کدگذاری گری مدولاسیون DBPSK را تولید کنید و آن را در ماتریس `grayMatrix` ذخیره کنید. بردار جدیدی متناظر با ماتریس `encodedTxBits` و با نام `symbolIndex` تولید نمایید و نگاشتی یک‌به‌یک بین سطرهاى ماتریس `encodedTxBits` و ماتریس `grayMatrix` برقرار کنید. به عبارتی سطر  $i$ ام بردار `symbolIndex`، شماره‌ی سطر  $i$  از `grayMatrix` را نشان می‌دهد که برابر با سطر  $i$ ام ماتریس `encodedTxBits` باشد.
  - تولید سمبل‌های ارسالی: با استفاده از تابع `constellation`، تمامی سمبل‌های ارسالی مدولاسیون DBPSK را تولید کرده و سمبل‌های متناظر با هر سطر بردار `symbolIndex` را تولید کنید. این سمبل‌ها را در بردار `symbolArray` ذخیره نمایید.
- شکل‌دهی پالس ارسالی: با استفاده از تابع `pulseModulation` نمونه‌های ارسالی مربوط به یک بسته‌ی داده را آماده‌ی ارسال کرده و حاصل را در بردار `txSamples` ذخیره نمایید. تابع شکل‌دهی پالس را مثلاً در نظر بگیرید.

## آزمایش ۴-۵: مدل‌سازی کانال

- افزودن تأخیر در کانال: برای شبیه‌سازی تأخیر در کانال به اندازه‌ی پارامتر `channelDelayInSample` به ابتدای بردار `txSamples` صفر اضافه نمایید. حاصل را درون بردار `txSamplesDelayed` قرار دهید. دقت نمایید که طول بردار افزایش می‌باید و می‌بایست در ادامه‌ی برنامه اثر این افزایش لحاظ شود. در این جا فعلاً مقدار تأخیر برابر با صفر قرار دهید.
- اعمال ابهام فاز کانال: مدولاسیون DPSK به تغییرات شدید فاز حساس است و بنابراین باید ابهام فاز ایجاد شده خیلی زیاد نباشد. ابهام فاز ایجاد شده برای سمبل‌ها را به صورت زیر مدل می‌کنیم  

$$\text{channelPhase} = 0 : \text{phase\_step} \text{phaseStep} : (nPacketSymbols-1) * \text{phase\_step} \text{phaseStep}$$
 پارامتر `phase_step phaseStep`، مقدار ابهام فاز ایجاد شده برای یک سمبل نسبت به سمبل قبلی را مشخص می‌کند. حال با استفاده از دستور `kron`، فاز تولید شده را برای کل نمونه‌های هر سمبل تکرار کنید. با ضرب کردن عبارت `exp(1j*channelPhase)` در `txSamplesDelayed`، ابهام فاز ایجاد شده را به سینگال ارسالی اضافه نمایید و حاصل را در `txSamples` ذخیره نمایید. در این جا `phase_step phaseStep` را برابر با ۱۰ درجه قرار دهید.
- شبیه‌سازی کانال با نویز سفید گاوسی
  - تعیین واریانس نویز بر اساس نسبت سیگنال به نویز ( $E_b/N_0$ ): ابتدا بر اساس خروجی تابع `constellation`، مقدار متوسط انرژی سمبل را به دست آورید (`symbolAverageEnergy`). حال با استفاده از متوسط انرژی سمبل، مقدار انرژی متوسط بیت را به دست آورده و برابر متغیر `bitAverageEnergy` قرار دهید. مقدار  $E_b/N_0$  را برابر با 10dB در نظر بگیرید (به عبارتی پارامترهای `snrMin` و `snrMax` برابر 10 تنظیم می‌شوند). سپس واریانس نویز را بر اساس نسبت  $E_b/N_0$  به دست آورید و آن را درون متغیر `noiseVariance` قرار دهید.

گام ۴. افزودن نویز به سیگنال: با استفاده از تابع `randn` یک بردار نویز مختلط با واریانس `noiseVariance` و ابعاد برابر با `txSamples` تولید نمایید و آن را `noiseSamples` نامگذاری نمایید. سپس این بردار را با بردار `txSamples` جمع نموده و حاصل را `txSamples_noisetxSamplesPlusNoise` بنامید.

## آزمایش ۵-۵: پیاده‌سازی گیرنده‌ی مدولاسیون DBPSK

در ادامه‌ی فایل آزمایش قبل می‌خواهیم گیرنده‌ی مدولاسیون DBPSK را پیاده‌سازی نماییم.

۱. دمدولاسیون یا آشکارسازی پالس: نمونه‌های ارسال شده به همراه نویز (`txSamplesPlusNoise`) را به عنوان نمونه‌های دریافتی درون بردار `rxSamples` قرار دهید. با استفاده از تابع `pulseDemodulation` با ورودی‌های نمونه‌های سیگنال دریافتی (`rxSamples`)، نام مدولاسیون (`modulation`)، مرتبه‌ی مدولاسیون (`M`)، نرخ نمونه‌برداری (`fs`)، تعداد نمونه‌های هر سمبل (`nSymbolsSamples`)، نام تابع شکل‌دهنده‌ی پالس (`pulseName`)، روش تشخیص قدرت سیگنال دریافتی در راستای سیگنال‌های پایه (`mode`) و شیوه‌ی آشکارسازی (`det_opt`)، سمبل‌های دریافتی (`rxSymbols`) و اندیس سمبل‌های آشکارشده (`detectedSymbolsIndex`) را به دست آورید.
۲. محاسبه‌ی خطای سمبل: با مقایسه‌ی تعداد اختلاف‌های بردارهای `detectedSymbolsIndex` و `symbolIndex` و محاسبه نسبت اختلاف این دو بردار به تعداد کل سمبل‌ها، خطای سمبل را به دست آورده و آن را در متغیر `ser` قرار دهید.
۳. تبدیل سمبل و بیت و تبدیل از کدگذاری گری به کدگذاری باینری: بیت‌های متناظر با سطرهای `detectedSymbolsIndex` را به دست آورید و آن را درون متغیر `detectedBits_gray` قرار دهید. با وجود این که در این جا عکس تبدیل گری مطرح نیست به منظور کلی ماندن برنامه بیت‌های گری را به کدگذاری باینری تبدیل نماید و آن را درون بردار `detectedBits` قرار دهید.
۴. محاسبه‌ی خطای بیت: با مقایسه‌ی تعداد اختلاف ماتریس‌های `detectedBits` و `txBit` و محاسبه نسبت تعداد اختلاف این دو ماتریس به تعداد کل بیت‌ها، خطای بیت را به دست آورده و آن را در متغیر `ber` قرار دهید.

## آزمایش ۵-۶: خواسته‌های کلی

۱. رسم نمودار نرخ خطای بیت: با تغییر نسبت سیگنال به نویز بین 0 تا 10dB نرخ خطای بیت را برای مدولاسیون‌های DBPSK به دست آورده و در یک نمودار رسم نمایید و آن را با خروجی تابع `berawgn` مقایسه نمایید. این کار را برای شکل موج مثلی انجام دهید.
۲. اثر تغییرات فاز بر روی عملکرد: با تغییر پارامتر `phaseStep`، تأثیر تغییرات فاز را بر روی عملکرد این سامانه ارزیابی نمایید.

## آزمایش ۵-۷: پیاده‌سازی سخت‌افزاری مدولاسیون DBPSK

۱. ارسال نمونه‌ها با استفاده از ADALM-PLUTO: بردار `txSamples` مربوط به مدولاسیون DBPSK (هذر اضافه شود) را با استفاده از دستور `transmitRepeat` به مدت زمان `dataTransferDuration` ثانیه به صورت پی‌درپی در فضا ارسال نمایید. این دستور داده‌ها را از طریق USB به رادیو نرم‌افزار ارسال می‌نماید و در حافظه‌ی سخت‌افزار ذخیره می‌نماید. فرستنده داده‌ها را از حافظه‌ی رادیو نرم‌افزار قرائت کرده و مدام ارسال می‌نماید. برای این منظور ابتدا می‌بایست پس از پیکربندی رادیو نرم‌افزار یک شیء فرستنده ایجاد نموده و فرکانس مرکزی آن را بر روی 2400MHz تنظیم کرده و بهره‌ی آن را برابر با 0dB تنظیم نمایید.
۲. دریافت سیگنال با استفاده از ADALM-PLUTO: ابتدا شیء مربوط به گیرنده را تعریف نمایید. فرکانس مرکزی آن را بر روی 2400MHz تنظیم کرده و بهره‌ی آن را به صورت Manual و برابر 20dB تنظیم نمایید. به علت اطمینان از دریافت یک بسته‌ی

کامل، تعداد نمونه‌های زمانی دریافتی را برابر با دو برابر تعداد نمونه‌های زمانی بسته در نظر بگیرید. به منظور دریافت داده، شیء گیرنده را فراخوانی نمایید و حاصل را درون متغیر **rxSamples** قرار دهید.

۳. دمدولاسیون و مشاهده‌ی منظومه‌ی سیگنالی: مشابه قبل عمل دمدولاسیون را انجام داده و منظومه‌ی سیگنالی مربوط به **rxSymbols** را رسم نمایید و مشاهدات خود را یادداشت نمایید. منظومه‌ی سیگنالی باید بتواند به صورت به لحظه به روز شود.



- [3] [1] R. W. Stewart, K. Barlee, L. Crockett, and D. Atkinson, *Software Defined Radio using MATLAB & Simulink and the RTL-SDR*. 2015.





# آزمایش ششم

## انتقال دیجیتال از درون کانال باند محدود AWGN

### اهداف آزمایش

در طول آزمایش‌های قبل فرض بر آن بود تمرکز بر روی آشکارسازی همدوس بوده است. در صورت وجود تأخیر کانال و تغییر فاز سیگنال دریافتی، ابتدا این فاز جبران و سپس آشکارسازی صورت می‌پذیرفت. در این آزمایش با مدولاسیون‌هایی آشنا می‌شویم که با وجود جبران نکردن فاز می‌توان عمل آشکارسازی را انجام داد که به این آشکارسازی، آشکارسازی ناهمدوس گفته می‌شود.

در انتهای این آزمایش دانشجو می‌بایست:

-  با مفهوم آشکارسازی ناهمدوس آشنا شود.
-  آشکارسازی ناهمدوس مدولاسیون FSK را انجام دهد.
-  با نحوه‌ی تولید و آشکارسازی مدولاسیون DBPSK آشنا شود.
-  با شبیه‌سازی کانال با فاز تصادفی آشنا باشد.
-  این نوع آشکارسازی را با سخت‌افزار پیاده‌سازی نماید.

### ابزارهای مورد نیاز

-  رایانه
-  نرم‌افزار MATLAB R2020b
-  رادیو نرم‌افزار ADALM-PLUTO
-  کابل کوکسیال
-  آنتن



## کانال باند محدود

در آزمایش‌های قبل کانال مورد استفاده دارای باند فرکانسی نامحدود فرض می‌شد. در واقع مدل کانال مورد استفاده در حوزه‌ی زمان به صورت تابع ضربه بود. در عمل کانال‌های مخابراتی دارای پاسخ زمانی غیرایده‌آل است و باعث ایجاد تداخل بین سمبلی می‌شود. تداخل بین سمبلی، شکلی از اعوجاج سیگنال است که هر سمبل با چندین سمبل دیگر تداخل پیدا می‌کند. این امر، یک اتفاق ناخواسته است که سمبل‌های قبلی به دلیل پهن شدن در حوزه‌ی زمان، هم‌چون نویز برای سمبل فعلی ظاهر می‌شوند و قابلیت اطمینان سیستم‌های مخابراتی را کاهش می‌دهند. وجود تداخل بین سمبلی در سیستم باعث ایجاد خطا در تصمیم‌گیری می‌شود. بنابراین لازم است که فیلترهای فرستنده و گیرنده را به گونه‌ای طراحی کنیم که تا حد ممکن از خطاهای ایجاد شده جلوگیری شود.

دسته‌ی مهمی از کانال‌ها غیرایده‌آل، کانال‌های باند محدود هستند که پاسخ فرکانسی آنها برای فرکانس‌های مشخصی برابر با صفر است. عبور سیگنال از این کانال‌ها منجر به حذف فرکانس‌های خارج از باند گذر کانال می‌شود. از طرف دیگر، ممکن است که فرکانس‌هایی که در محدوده‌ی باند گذر کانال هستند نیز تضعیف شوند.

کانال‌های باند محدود هم در مخابرات سیمی و هم در مخابرات بی‌سیم وجود دارند. در بسیاری از موارد، این محدود شدن باند می‌تواند عمدی باشد. به عنوان مثال اگر بنا باشد چندین کاربر به صورت همزمان از یک محیط برای انتقال داده استفاده کنند، می‌توان به عنوان نمونه باند فرکانسی کاربران را از یکدیگر ایزوله نمود. محدود بودن باند می‌تواند به دلیل مشخصات فیزیکی محیط انتقال نیز باشد. به عنوان مثال ممکن است که یک کابل دارای فرکانس قطع مشخصی باشد و اجازه‌ی ارسال سیگنال در فرکانس‌های بالاتر را ندهد. معمولاً سیستم‌های مخابراتی که دارای کانال‌های باند محدود هستند از شکل‌دهی پالس برای جلوگیری از ایجاد تداخل استفاده می‌کنند.

## الگوی چشم (Eye Pattern)

خروجی فیلتر ارسالی در سامانه‌ی مخابراتی با مدولاسیون PAM، PSK یا QAM را می‌توان به صورت زیر بیان نمود.

$$v(t) = \sum_{n=-\infty}^{+\infty} a_n g_T(t - nT)$$

هم‌چنین خروجی کانال که در واقع سیگنال دریافتی در دمولاتور است نیز می‌توان به صورت زیر بیان کرد.

$$r(t) = \sum_{n=-\infty}^{+\infty} a_n h(t - nT) + n(t)$$

در رابطه‌ی فوق  $h(t) = c(t) * g_T(t)$  است که در آن  $c(t)$  پاسخ ضربه‌ی کانال،  $g_T(t)$  پاسخ ضربه‌ی فیلتر ارسالی و  $n(t)$  نیز یک تابع نمونه از یک فرآیند نویز گاوسی سفید جمع شونده می‌باشد.

برای طراحی یک فیلتر ارسالی باند محدود ابتدا طراحی در شرایطی که هیچ اعوجاج کانالی وجود ندارد صورت می‌گیرد. سپس اثر اعوجاج کانال مورد بررسی قرار می‌گیرد. از آن‌جا که  $H(f) = C(f)G_T(f)$  شرط انتقال بدون اعوجاج آن است که پاسخ فرکانسی  $C(f)$  کانال دارای دامنه‌ی ثابت و فازی خطی در کل پهنای باند سیگنال ارسالی باشد. به عبارتی به صورت رابطه‌ی زیر باشد.

$$C(f) = \begin{cases} C_0 e^{-j2\pi f t_0} & |f| \leq W \\ 0 & |f| > W \end{cases}$$

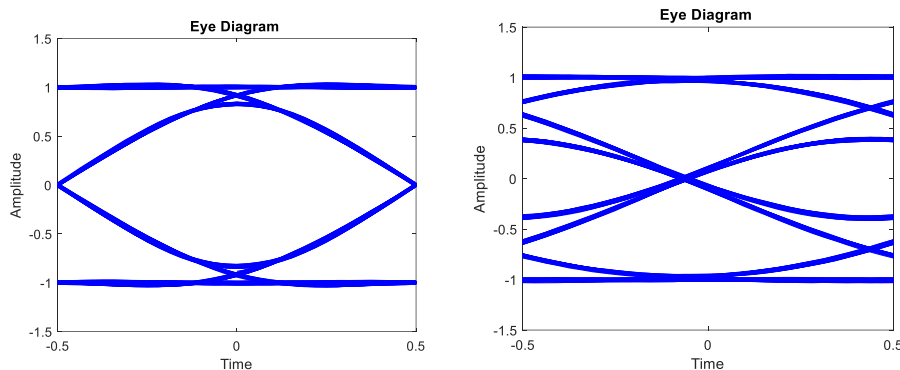
در عبارت فوق  $W$  پهنای باند در دسترس کانال،  $t_0$  یک تأخیر دلخواه محدود و  $C_0$  نیز یک ضریب بهره‌ی ثابت است. برای سادگی می‌توان فرض نمود  $C_0 = 1$  و  $t_0 = 0$  است. در شرایطی که کانال بدون اعوجاج است برای  $|f| \leq W$  خواهیم داشت  $H(f) = G_T(f)$  و برای  $|f| > W$  صفر خواهد بود. در نتیجه فیلتر منطبق دارای پاسخ فرکانسی  $H^*(f) = G_T^*(f)$  بوده و خروجی آن در زمان‌های نمونه‌برداری متناوب  $t = mT$  به صورت زیر است.

$$y(mT) = x(0)a_m + \sum_{n \neq m} a_n x(mT - nT) + v(mT) \equiv y_m = x_0 a_m + \sum_{n \neq m} a_n x_{m-n} + v_m$$

در عبارت فوق  $x(t) = g_T(t) * g_R(t)$  و  $v(t)$  نیز خروجی فیلتر منطبق به فرآیند AWGN ورودی  $n(t)$  می‌باشد.

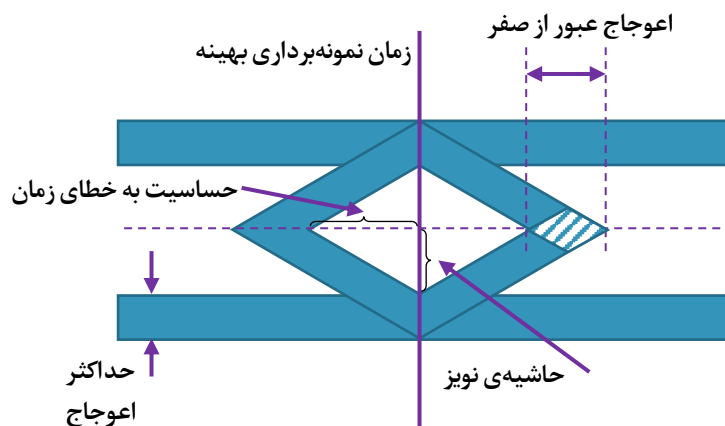


در رابطه‌ی فوق عبارت  $\sum_{n \neq m} a_n x_{m-n}$  تداخل بین سمبلی (ISI) را بیان می‌کند. مقدار این تداخل بین سمبلی و نویز حاضر در سیگنال دریافتی را می‌توان بر روی یک نوسان‌نگار مشاهده نمود. برای این منظور می‌بایست سیگنال دریافتی را بر روی محور عمودی نشان داد و نرخ جاروب محور افقی را نیز برابر با  $1/T$  قرار داد. نمایش حاصل بر روی نوسان‌نگار را الگوی چشم گویند که علت آن شباهتی است که با چشم انسان دارد. نمونه‌هایی از دو الگوی چشم در شکل 14 نشان داده شده است.



شکل 14 نمونه‌های از نمودار چشمی

تداخل بین سمبلی باعث بسته شدن چشم می‌شود و به عبارتی منجر به کاهش حاشیه‌ای که نویز جمع‌شونده ایجاد خطا می‌کند می‌شود. شکل 15 تأثیر تداخل بین سمبلی را بر روی کاهش باز بودن چشم نشان می‌دهد. می‌بایست توجه کرد که تداخل بین سمبلی لحظات گذر از صفر را دچار اعوجاج کرده و باعث کاهش باز بودن چشم می‌شود. در نتیجه این سامانه به خطای همگام‌سازی حساسیت بیشتری داشته و حاشیه‌ی کمتری در برابر نویز جمع‌شونده نشان می‌دهد.



شکل 15 شمای اثر تداخل بین سمبلی بر روی مقدار بازی الگوی چشم

## شرح آزمایش

### آزمایش ۱-۶: مقداردهی‌های اولیه

در این آزمایش بنا داریم تعدادی بسته‌ی داده را برای ارسال از طریق مدولاسیون 2PAM آماده نماییم. از آن‌جا که در این آزمایش بناست این فرستنده با استفاده از رادیو نرم‌افزار ADALM-PLUTO پیاده‌سازی شود، می‌بایست با استفاده از پارامترها و ملاحظات عملی، این کار انجام شود.

پارامترهایی که در این پیاده‌سازی اهمیت دارد شامل نرخ نمونه‌برداری ( $f_s$ )، تعداد نمونه‌های هر سمبل ( $nSymbolSamples$ )، تعداد سمبل‌های ارسالی در بسته‌ی داده ( $nPacketSymbols$ ) و مدت زمان ارسال داده ( $dataTransferDuration$ ) می‌باشد. در این آزمایش مقدار پارامترهای مورد نیاز به صورت جدول 3 می‌باشد.

جدول 5 پارامترهای آزمایش ۷

| پارامتر | $f_s$ | $nSymbolSamples$ | $nPacketSymbols$                     | $dataTransferDuration$ |
|---------|-------|------------------|--------------------------------------|------------------------|
| مقدار   | 10MHz | 32               | شبیه‌سازی: ۱۰۰۰۰۰<br>سخت‌افزار: ۱۰۰۰ | ۱۰۰ ثانیه              |

دقت نظر داشته باشید که در این آزمایش نیز تنها می‌خواهیم برنامه‌هایی که در آزمایش قبل نوشته شده است را تکمیل نماییم.

### آزمایش ۲-۶: پیاده‌سازی مدولاسیون 2PAM

۱. شبیه‌سازی مدولاسیون 2PAM: برای پیاده‌سازی این مدولاسیون، همانند آزمایش ۴ یا ۵ عمل نمایید. احتمال خطای بیت را برای نسبت  $\frac{E_b}{N_0}$  برابر با 10dB به دست آورید. این عمل را برای دو شکل پالس مستطیلی و Root Raised Cosine ( $\beta = 0.9$ ) و  $spanInSymbol = 6$  انجام دهید. (حالت عملکردی دمدولاتور را بر روی فیلتر منطبق تنظیم نمایید.)

۲. رسم نمودار چشمی: با استفاده از دستور **eyediagram** نرم‌افزار MATLAB نمودار چشمی مدولاسیون 2PAM را برای دو شکل پالس گفته شده و در دو نسبت  $E_b/N_0$  برابر با 10dB و 40dB رسم نمایید. پارامترهای بر روی شکل 15 را به دست آورید.

### آزمایش ۳-۶: مدل‌سازی کانال باندمباریک

۱. تولید یک فیلتر FIR: با استفاده از دستور **fir1** نرم‌افزار MATLAB یک فیلتر با پهنای باند 300kHz و با تعداد ۱۰۰ تپ تولید نمایید. پاسخ فرکانسی و پهنای باند نویز فیلتر را به دست آورید و با پهنای باند سیگنال ارسالی مقایسه نمایید.

۲. کانال باندمحدود: پس از اعمال تأخیر و اعمال ابهام فاز، سیگنال ارسالی حاصل را با استفاده از عمل کانولوشن از فیلتر FIR مرحله‌ی قبل عبور دهید. مجدد نمودار چشمی را در دو نسبت  $E_b/N_0$  برابر با 10dB و 40dB رسم نمایید. احتمال خطای بیت را نیز به دست آورید. این عمل برای هر دو شکل پالس گفته شده انجام شود.

### آزمایش ۴-۶: پیاده‌سازی کانال باندمحدود

۱. کار با ADALM-PLUTO: موارد گفته شده را با استفاده از رادیو نرم‌افزار ADALM-PLUTO نیز انجام دهید.

### آزمایش ۵-۶: خواسته‌های کلی

۱. اثر طول شکل پالس root raised cosine: با تغییر پارامتر **spanInSymbol** تأثیر آن را بر روی نمودار چشمی و خطای بیت مدولاسیون 2PAM با شکل پالس root raised cosine را مشاهده نمایید.



- [1] R. W. Stewart, K. Barlee, L. Crockett, and D. Atkinson, *Software Defined Radio using MATLAB & Simulink and the RTL-SDR*. 2015.

